

ניתוח ועיצוב מערכות מידע

תרשים מחלקות — Class Diagram

הרצאה 6-7

עודכן: 19.08.2026

1 יסודות — מחלקה ותרשים

2 אנטומיה של מחלקה

3 קשרים בין מחלקות

4 מניתוח לעיצוב טוב

5 תרגול וסיכום

מה קורה בפרויקט בנקודה הזו

סוף שלב הניתוח — מסירה מצוות הניתוח העסקי לצוות הטכני

צוות ניתוח עסקי (BA)

סיים לענות על מה המערכת צריכה לעשות — ומוסר חבילת תוצרים:

דרישות SMART הרצאה 3

קטלוג חוקים עסקיים (BR-n)
הרצאה 3 · חוק שטרם הוחלט = החלטה שעוד חסרה לעיצוב

תרחישי שימוש (Use Case) הרצאה 4-5

תרשימי תהליך (BPMN) הרצאה 2

דרישות לא-פונקציונליות עומס, זמינות, אבטחה, ביצועים



מסירה

סקירה משותפת ואישור התוצרים — לא "זריקה מעבר לגדר". ה-BA נשאר זמין להבהרות לאורך העיצוב.

צוות טכני — עיצוב

עונה על איך המערכת תיבנה. לא איש אחד: כמה בעלי תפקיד עובדים במקביל על אותו קלט.

היום

מודל הנתונים

איזה מידע המערכת מחזיקה ואיך הוא מאורגן — תרשים מחלקות מנתח מערכת (System Analyst)

מסד הנתונים

טבלאות, מפתחות, אינדקסים, גיבוי ושחזור
DBA · מהנדס נתונים

ממשק המשתמש

מסכים, זרימות עבודה, נגישות ושפה ויזואלית
UX / UI

ארכיטקטורה ותשתית

שכבות, אינטגרציה למערכות קיימות, אבטחה, עמידה בעומס
ארכיטקט · DevOps

כל הכיסאות עובדים על אותה חבילת דרישות — היום אנחנו יושבים באחד מהם: מודל הנתונים.

עיצוב

Business Analyst = BA · Business Process Model and Notation = BPMN · מידול וסימון תהליכים עסקיים · Software Development Life Cycle = SDLC · מחזור פיתוח תוכנה · Specific, Measurable, Achievable, Relevant, Traceable = ספציפי, מדיד, בר-השגה, רלוונטי, עקיב · SMART · אנליסט/ית עסקית · אבטחה, עמידה בעומס, נגישות ושפה, ויזואלית

היכן אנחנו ב-SDLC

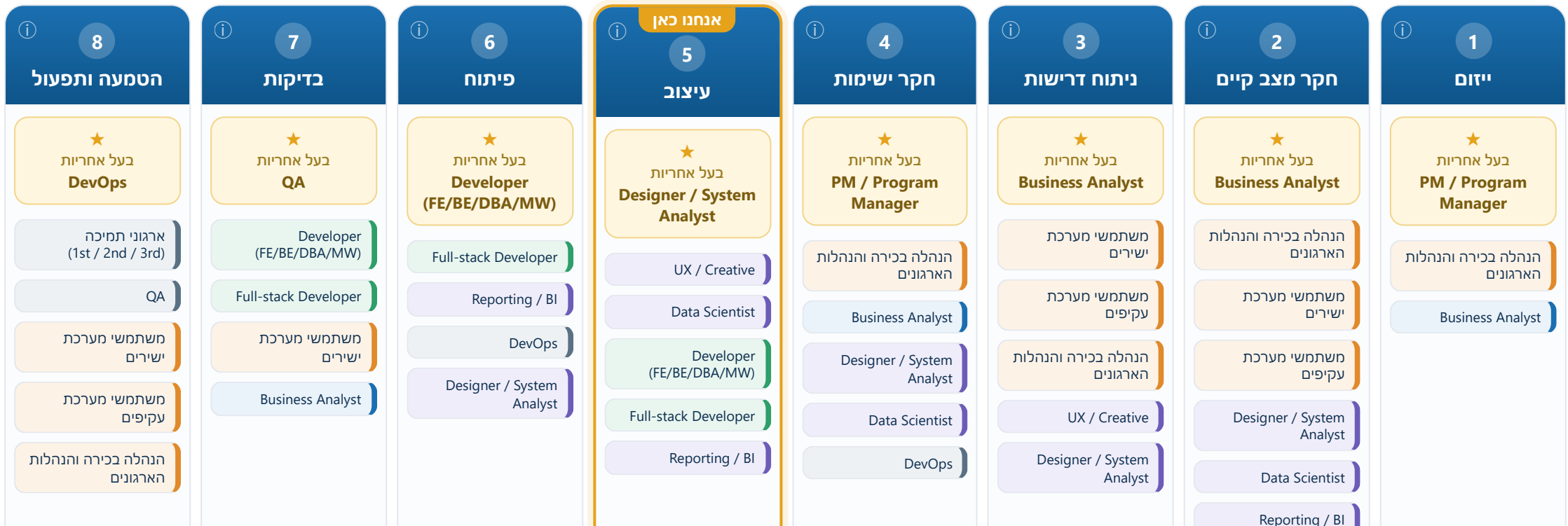
שלב עיצוב — כל שלב במחזור החיים ובעלי התפקידים המעורבים בו · כאן עוברים ממה לאיך

שלבי ה-SDLC — מי מוביל ומי משתתף בכל שלב

מיפוי בעלי התפקידים והאקטורים לאורך מחזור החיים של מערכת המידע

← הזמן מתקדם מימין לשמאל

לחצו על שלב או על תפקיד לקבלת הסבר קצר.



עיצוב

מחלקות ואובייקטים

Class = תבנית · Object = מופע

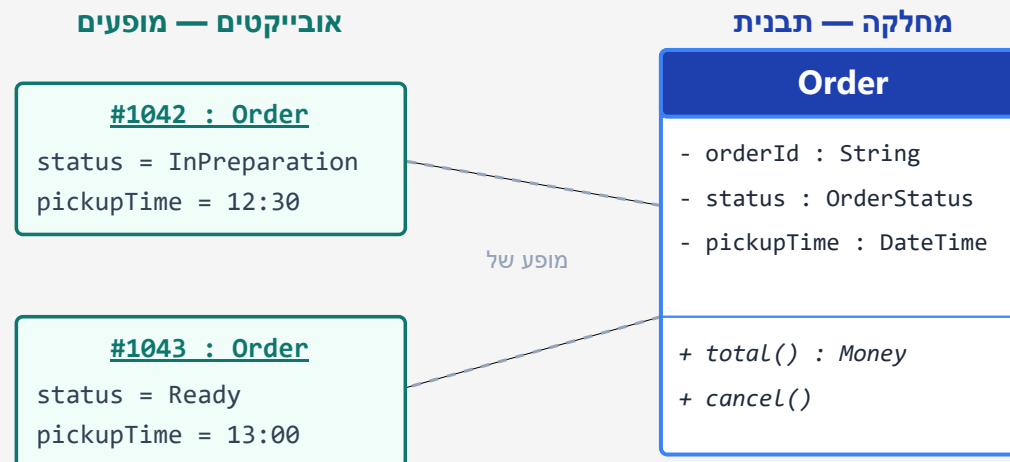
מחלקה = תבנית

הגדרה כללית של קבוצת אובייקטים בעלי אותן תכונות ופעולות. Class — כמו טופס ריק, נכתב פעם אחת.

אובייקט = מופע

Object קונקרטי בודד של המחלקה, עם ערכים ממשיים — הזמנה מסוימת של סטודנט מסוים.

מחלקה אחת — Order — הרבה מופעים



בעבודה עם AI

אפשר לבקש מ-AI למנות מתוך המפרט את המחלקות המועמדות (התבניות) — לא את האובייקטים הבודדים.

התרשים ממדל מחלקות (תבניות) — לא אובייקטים בודדים.

מהו תרשים מחלקות

המבנה הסטטי של המערכת — domain model

הגדרה

המבנה הסטטי של המערכת: המחלקות שהמערכת מנהלת והקשרים ביניהן. תרשים תרחישי שימוש מתאר התנהגות (מה) — תרשים המחלקות מתאר מבנה ונתונים (איך בנוי).

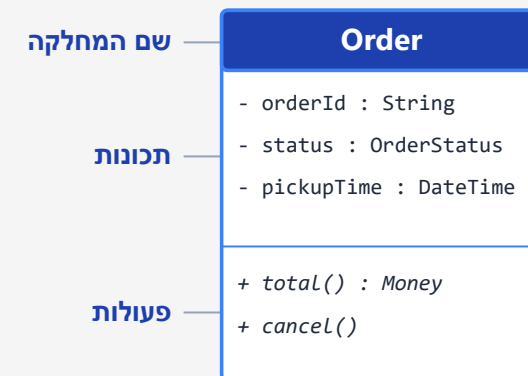
למה זה שימושי

- אוצר מילים משותף — שפה אחת לכל בעלי העניין על המחלקות של המערכת.
- השלד שממנו נגזרים בסיס הנתונים והקוד — גזירה, לא העתקה: המודל קובע איזה מידע קיים; איך הוא נשמר בפועל זו החלטת עיצוב בפני עצמה.

שכבת הנתונים 🗄️

זה גם מה ש-AI קורא כדי לייצר את שכבת הנתונים והפעולות — מודל ברור פנימה, קוד נכון החוצה.

שלושה חלקים לכל מחלקה



תוצר מרכזי של שלב העיצוב: מ"מ המערכת עושה" אל "איך בנוי המידע שלה".

use cases מ-למודל מחלקות

ה-AI מנסח מודל בשניות — ואנחנו מאתגרים אותו

מדביקים ל-AI את תרחישי השימוש והדרישות (הרצאה 4-5) — ומקבלים טיוטת מודל מחלקות עם קשרים ראשוניים, מיד.

⚠️ נראה שלם — אבל ה-AI נופל שוב ושוב

- ✗ הופך דוח מכירות / קבלה למחלקה — זה פלט מחושב שלא נשמר (מבחן ההתמדה — נרחיב בהמשך ההרצאה).
- ✗ ממציא מחלקות שהוצאו מהגבול — משלוח (מחוץ ל-scope).
- ✗ מנחש קרדינליות במקום לברר אותה.
- ✗ בונה תת-מחלקות שאין להן הצדקה — "הזמנה ממתינה" ו"הזמנה מוכנה" אינן סוגים של הזמנה, אלא סטטוס של אותה הזמנה.

🤖 מה ה-AI מפיק

רשימת מחלקות מועמדות מתוך אותה דוגמה רצה:

סטודנט הזמנה שורת הזמנה פריט תפריט תשלום עמדה קפיטריה

ומעליהן — הצעת קשרים וקרדינליות ראשונית. נראה שלם ומשכנע.

ההרגל שנקבע כאן

המודל הוא משטח האימות שלך. על כל טיוטה שאל: מה הומצא? מה חסר? מה הונח?

עיצוב

מחלקה — התיבה ושלושת החלקים

כל מחלקה = מלבן ובו שלושה תאים: שם · תכונות · פעולות

מחלקה = תיאור של קבוצת אובייקטים בעלי תכונות, פעולות וקשרים דומים.

Order

- orderId : String
- pickupTime : DateTime
- status : OrderStatus

+ total() : Money
+ cancel()
+ addItem(item : MenuItem, qty : int) : OrderLine
+ getStatus() : OrderStatus
+ setStatus(newStatus : OrderStatus)

שלושת החלקים

- 1 שם המחלקה — בראש התיבה, על הרקע הכחול
- 2 תכונות (Attributes) — הנתונים שהמחלקה מחזיקה על כל אובייקט
- 3 פעולות (Operations) — מה המחלקה יודעת לעשות, נכתבות בכתב נטוי

שם המחלקה

איך נראה שם תקין — ולמה הוא תמיד באנגלית

שם מחלקה הוא מזהה — הוא יהפוך לשם טבלה ולשם מחלקה בקוד. לכן יש לו כללי כתיבה, והוא לא טקסט חופשי.

אותו רעיון, שני ניסוחים

למה	✓ תקין	✗ שגוי
ביחיד — המחלקה היא התבנית לאובייקט אחד	Order	orders
בלי רווח; כל מילה נפתחת באות גדולה	MenuItem	menu-item
בלי קו תחתון או מקף	OrderLine	order_line
שם עצם — פעולה היא operation בתוך המחלקה	Order	PlaceOrder
מזהים נכתבים באנגלית — אין שמות טבלאות ומחלקות בעברית	Order	הזמנה

✓ אותו היגיון לתכונות ולפעולות
גם הן באנגלית — רק שהן נפתחות באות קטנה (lowerCamelCase): orderId · pickupTime · addItem() · getStatus() · מחלקה מתכונה.

כללי מתן שם

- מתחיל באות גדולה
- UpperCamelCase — כל מילה באות גדולה, בלי רווח ובלי מקף
- ביחיד — מחלקה מתארת אובייקט אחד, לא אוסף
- שם עצם, לא פעולה
- באנגלית — תמיד, גם כשכל השאר בעברית

💡 למה זה לא קפריזה
התרשים נקרא גם על ידי מפתח או AI שלא דובר עברית — ובסוף הוא הופך לשם טבלה ולשדה ב-API.

מדברים על ההזמנה בעברית — וכותבים Order בתרשים. זו העבודה בשטח.

UML מול שפת תכנות

התרשים הוא הסימון — השפה היא המימוש, והמעבר ביניהם אינו אוטומטי

כל מה שנלמד בהרצאה הוא תקן UML — ולא שייך לאף שפה

UML נוצר כדי שמהנדס יוכל לתאר מערכת בלי להתחייב ל-Java, ל-Python או ל-SQL. לכן שם הטיפוס בתרשים הוא שם שאנחנו בוחרים, לא מילה שמורה בשפה.

איפה השפה לא עוקבת

Money אינו טיפוס בשום שפה

זה טיפוס תחום שאנחנו הגדרנו. מותר וכדאי — התרשים מדבר בשפת העסק.

הורשה מרובה

קיימת ב-UML וב-C++ ;ב-Java וב-C# רק דרך interface.

הרכבה וצבירה

אין להן מילה שמורה באף שפה — הן הופכות לכלל מחיקה ולבדיקות בקוד ובמסד הנתונים.

אותו מרכיב במודל, חמישה יעדים

SQL	Python	Java	C# הפרויקט שלכם	בתרשים (UML)
VARCHAR(n)	str	String	string	String
TIMESTAMP	datetime	LocalDateTime	DateTime	DateTime
NUMERIC(10,2)	Decimal	BigDecimal	decimal	Money
View / הרשאות	מוסכמה בלבד — אין אכיפה	private	private	- private
טבלה עם מפתח זר	list[Phone]	List<Phone>	List<Phone>	Phone [0..*]

השורה האחרונה היא קשר ולא תכונה — ולכן הריבוי בה חוקי. בקוד הוא הופך לאוסף של אובייקטים, ובמסד הנתונים לטבלה עם מפתח זר — אין שם רשימה בכלל.

המודל לא נכתב עבור שפה מסוימת — אל תצמצמו אותו למגבלות של שפה בשלב הניתוח.

Attributes — תכונות

הנתונים שהמחלקה מחזיקה על כל אובייקט

```
- name : Type = default {constraint}
```

רק השם חובה — כל שאר המרכיבים לפי הצורך

Order

- orderId : String
- createdAt : DateTime
- pickupTime : DateTime
- pickupCode : String
- status : OrderStatus

+ total() : Money ...

מרכיבי התכונה

- שם — באות קטנה, camelCase

- טיפוס (dataType) — אחרי הנקודתיים: String, DateTime, Money...

- ברירת מחדל — ערך התחלתי אחרי =

- אילוצים — תמיד בסוגריים מסולסלים { }

- נראות — - / # / + (בהרחבה בשקופית הנראות)

מזהים — למי יש, ולמי אין

מזהה נכנס לתרשים רק אם העסק משתמש בו — לא כי הטבלה צריכה מפתח

המבחן: האם מישהו מחוץ למערכת מפנה לדבר הזה בעצמו? אם אומרים בקול "הזמנה 1042" — יש מזהה. אם אומרים "השניצל שבהזמנה 1042" — אין.

✗ אין מזהה משלו

OrderLine — אף אחד לא אומר "שורה 3". אומרים "השניצל בהזמנה 1042"

Station — מזוהה בתוך הקפיטריה שלה: "העמדה החמה"

Cafeteria — מזוהה בשם ובמיקום

ולא ממציאים להן מזהה רק כדי שיהיה.

✓ יש מזהה עסקי

Student · studentId — מודפס על כרטיס הסטודנט

Order · orderId, pickupCode — הסטודנט מקריא אותו בדלפק

MenuItem · itemId — קוד הפריט בקטלוג של הקפיטריה

Payment · paymentId — מופיע בקבלה ובאישור החיוב

⚠ ומפתח זר אינו תכונה

אל תוסיפו studentId ל-Order. מי הזמין — זה הקשר בין המחלקות (חלק ג'). המפתח הזר מופיע רק בטבלה.

🏠 אז איך מעדכנים אותן במסד הנתונים?

לאובייקט בזיכרון יש זהות בחינם — ההפניה עצמה. לשורה בטבלה אין: כדי לעדכן צריך לדעת לאן לפנות.

כדי להוסיף שורה לא צריך מזהה: כדי לעדכן או למחוק — כן. לכן למחלקה בלי מזהה עסקי המסד ממציא מפתח משלו — והוא נשאר במסד, לא בתרשים.

בזיכרון לאובייקט יש זהות בחינם; לשורה במסד צריך כתובת — ומשם נולד המזהה.

שתי דוגמאות — מהמחלקה לטבלה

אותה שאלה על שתי מחלקות: מאיפה מגיע המפתח של הטבלה?

OrderLine ✗ — אין לה מזהה

OrderLine	
- quantity : int	
- unitPrice : Money	
- notes : String	
אין שורת מזהה — וזה תקין	

↓
הופכת לטבלה

OrderLines	
lineId	המסד המציא אותו
orderId	מפתח זר
itemId	מפתח זר
quantity · unitPrice · notes	

לטבלה חייב להיות מפתח כדי שאפשר יהיה לעדכן שורה — אז המסד מייצר אותו.
lineId נשאר בטבלה בלבד ולא חוזר לתרשים: הוא לא עובדה עסקית.

Order ✓ — יש לה מזהה עסקי

Order	
- orderId : String	
- pickupCode : String	
- status : OrderStatus	

↓
הופכת לטבלה

Orders	
orderId	מפתח ראשי
pickupCode	
status	
studentId	מפתח זר — רק כאן

המזהה כבר היה בתרשים, והוא פשוט הופך למפתח הראשי. לא המצאנו כלום.
studentId לעומת זאת לא בתיבה — בתרשים הוא הקשר לסטודנט.

דוגמאות ואילוצים

איך מפרטים ערך של תכונה — ואיך כותבים עליו חוק

דוגמאות ערך

`price : Money` טיפוס בלבד

`price = 0` ערך התחלתי

`price : Money = 0` טיפוס + ברירת מחדל

`price {≥ 0}` אילוץ

אילוצים { }

אילוץ נכתב תמיד בסוגריים מסולסלים ומגדיר חוק על הערך:

`price {≥ 0} · quantity {> 0}`

יש גם מילות מפתח תקניות: `{readOnly}` · `{unique}` — לצד אילוץ חופשי משלכם.

אילוץ מורכב אפשר לתלות ב-Note (פתק):

`{ quantity > 0 }`

כשהמודל נפגש עם המציאות

טיפוס ואילוץ הם הבטחות — ואת אלה שנשברות משלמים בייצור

1 · הטיפוס הלא נכון



~~plateNumber : int~~

+ plateNumber : String

- אפסים מובילים נעלמים: 03-765-464 יהפוך ל-3
- בגרמניה יש אותיות: WOB ZK 295
- אף אחד לא מחבר שתי לוחיות רישוי

הכלל: אם לא מחשבים איתו — הוא לא מספר. גם ת"ז, טלפון ומיקוד הם String.

2 · אילוץ סגור מדי



~~machineColor = "White" {"White", "Black"}~~

- האילוץ קבע שיש שני צבעים בעולם
- ואז יצא הדגם הזה — והמערכת לא מסוגלת לקלוט אותו
- מי שמתקן את זה משנה קוד, לא נתונים

הכלל: רשימה שהעסק עשוי להרחיב אינה אילוץ — היא נתונים. אילוץ שומרים למה שבאמת לא ישתנה (≥ 0).

כל אילוץ שאתם כותבים — שאלו קודם: מה יקרה כשהמציאות תפר אותו?

Enumeration — רשימת ערכים סגורה

עד עכשיו כתבנו String, DateTime : — כאן מגדירים טיפוס משלנו, שהערכים החוקיים שלו ידועים מראש

אותו תנאי — שלוש דרכים לכתוב אותו

```
if (day == 1)
```

✗ מה זה 1? מספר קסם. צריך לזכור את הטבלה או לחפש אותה.

```
if (day == "Sunday")
```

✗ קריא — אבל אף אחד לא בודק. "Sundey" יתקמפל מצוין, וייכשל אצל הלקוח.

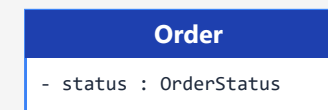
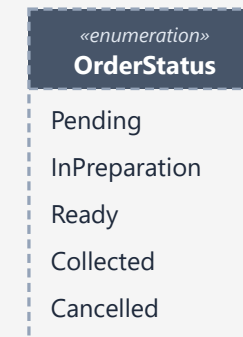
```
if (day == DayOfWeek.Sunday)
```

✓ קריא וגם נבדק. טעות כתיב לא מתקמפלת, וסביבת הפיתוח משלימה את הערכים החוקיים.

מתחת למכסה המנוע 💡

ב-C# הקומפיילר ממפה כל שם למספר — כותבים Sunday, ומשווים מספרים. לא בכל שפה כך:
ב-Java וב-Python ערך enum הוא אובייקט.

הסימון



אין קו בין השתיים!

טיפוס התכונה הוא כל הקישור

מה תקני ומה שלנו: תקני — «enumeration», רשימת הערכים, ובלי קו אל המחלקה. הרקע האפור והמסגרת המקווקוות הם מוסכמת התצוגה שלנו, לא דרישה — בכלי אחר אותו enum ייראה בתיבה רגילה.

שם קריא — בלי לוותר על בדיקה. זה כל הרווח.

enum או מחלקה?

השאלה המכריעה אינה כמה ערכים יש — אלא מי מנהל את הרשימה

שתי שאלות, בסדר הזה:

1. האם הקוד מתפצל לפי הערך (switch / if)? אם כן — הקוד חייב להכיר את הערכים.

2. מי בעל הרשימה — מפתח, או משתמש עסקי שרוצה לשנות אותה בעצמו?

enum — כשהרשימה של המפתח

- סגורה ויציבה — משתנה פעם בשנים, אם בכלל
- הקוד מתפצל לפיה: מה שקורה ל"מוכן" שונה ממה שקורה ל"בוטל"
- שינוי בה הוא שינוי קוד — וזה בסדר, כי ממילא צריך לכתוב את ההתנהגות החדשה
- האורך הוא סימפטום, לא כלל — עשרות ערכים כמעט תמיד אומרים "טבלה", כי אף קוד לא מתפצל ל-80 מקרים

דוגמאות: OrderStatus · DayOfWeek · סוג תשלום

מחלקה — כשהרשימה של העסק

- העסק משנה אותה: מוסיפים, מוציאים משימוש, משנים שם
- לערכים יש נתונים משלהם — סדר תצוגה, פעיל/לא פעיל, תרגום, תיאור
- הקוד לא מתפצל לפיה — רק מציג, שומר ומסנן

דוגמאות: קטגוריות פריטים · סיבות ביטול · מגדר (ארגונים מרחיבים את הרשימה עם הזמן)

⚠ ואי אפשר מסך ניהול ל-enum

ה-enum מהודר לתוך התוכנה. הוספת ערך = שינוי קוד ← בנייה ← בדיקות ← גרסה חדשה בייצור. ולכן: אם כל מה שצריך זו רשימה נפתחת שמישהו בארגון עשוי להרחיב — זו טבלה עם מסך ניהול, כלומר מחלקה. מסך ניהול מחייב טבלה; טבלה מחייבת מחלקה.

אם משתמש צריך להוסיף ערך ביום שלישי בצהריים — זה כבר לא enum.

מגדר — המקרה שמלמד הכול

אותה תכונה, שלוש תשובות — אחת נשברת תמיד, ובין השתיים האחרות תלוי במערכת

שלוש אפשרויות — אחת פסולה, שתיים תלויות במערכת

- isMale : Boolean

✗ פסול תמיד — שתי אפשרויות, ומי שאינו אחת מהן פשוט לא ניתן להזנה. וגם: מה זה ?false

- gender : Gender «enumeration»

⚖ כשהמערכת רק מציגה או אוספת לסטטיסטיקה — הקוד לא מתפצל, והרשימה זזה אחת לשנים.

Gender — 0..1 (מחלקה + טבלה)

⚖ כשהרשימה מנוהלת מבחוח — טפסים ורגולציה: מסך ניהול, סדר תצוגה, תרגום, השבתת ערך ישן.

enum למגדר — בעד ונגד

✓ בעד

- פשוט: אין טבלה, אין קשר, אין מסך ניהול לתחזק
- הערך מוגן — אי אפשר להזין ערך שאינו ברשימה
- קריא בקוד: Gender.Other, לא מחרוזת חופשית
- מספיק לחלוטין אם רק מציגים ומסכמים

✗ נגד

- ערך חדש = גרסה חדשה: קוד, בדיקות, פריסה
- אין סדר תצוגה, אין תרגום, אין תיאור לערך
- אין השבתה — ערך שיצא משימוש נשאר בקוד
- הבעלים האמיתי הוא רגולציה, לא המפתח

הראיות: הרשימה משתנה — ומגורמים שמחוץ לצוות

לראשונה באירופה: לא חובה להגדיר מין תינוק

מיום שישי תאפשר גרמניה לתינוקות שיוולדו עם מאפיינים של שני המינים להירשם בתעודת הלידה לא כזכר ולא כנקבה. עם כניסת החוק לתוקף יהיו בדרךון גרמני 3 שדות: M עבור זכר, F עבור נקבה X-4 עבור מין לא מוגדר

Recommend 142

פורסם: 11:15, 30.10.13 AFP

גרמניה, 2013 — נוסף ערך שלישי לטופס רשמי

פייסבוק — רשימה פתוחה, נערכת בלי לגעת בקוד

enum או מחלקה? — חידון

לחצו על כרטיס כדי להפוך אותו. קודם מחליטים בכיתה — אחר כך בודקים

בכל כרטיס – רשימת ערכים. שלוש השאלות: האם הקוד מתפצל? מי בעל הרשימה? והאם לערכים יש נתונים משלהם?

הזז מלל הזז מלל X

קצו, (אזל און שטופנדיג) חורחוס חוואסא
הצורח און — מלשחה ששעל ששק שי יאצואל
(יא קלח).

הקלח-ח-ח — שדח זוק אליוס? ידיתא אוסו
 מינוחן יוא מיוסלס מאו. מייסא מלא יבא, רחסו
 הקלח. bordhMtnemyg9 חשורו יוא — מיוס

University of Cambridge
Cambridge University

דוקה. יניחש מוי קיסוי אל' להגה קא — שטט שוהך
השטט הגהט ולישאו, (החיתום חושש) לעהטת

«enumeration» DayOfWeek
(built-in C#)

הקדמה

חדש מיכאל; וועגן חנוכה קומט היינטיקע לחנוכה.
איצט קען דאס. לייט-אלעלישט האבן

```

        γρογαςC
{ evitcAzi , ρεβrOγδIqzib , εμαν }
        γρογαςC I - *.0 μεtIunεM

```


יְיָנוּמֵי הַחֹקֶשׁ הַזֶּה — עָרַץ לֹא יֵשֶׁת לַעֲשֵׂת זֶהָ
הַמִּשְׁמָרָה יֵשֶׁת מִתְחַסֶּה. יִשְׁטוּבֵי חֲנוּשׁ

OrderStatus «enumeration»
Pending, InPreparation, Ready,
Collected, Cancelled

יזלז

פונקטור $\rightarrow$ חיסול מיסוס זוקסו, זבלס חיווה

שיימיסרעל → מיטוואך חצות AJN האנדל
 {מאמע, פאטער} קאלירן ← מיטל

שפוט

מחמת → (זיכפתה יפ'ל fi) זוכב חושבב חושבחה

הקלטה → קטע מוואציה זיגאל נאמל
 Role { name, permissions }

 חשפאם יזרח

פחמון → הקייטסיטטס' מיססוא וא, מיאצו רך
זיס

הקצתם → הלחננו חזיתו — מוצי
המאמר — הילגורו מיספט לוח היה המרשח

{ enitive } { mame }

הערה

— לצפתה אל זוקה; וְנִזְחָה לֹא חֻבֵּיס מִיִּסּוּם קִסְמָה.
הַקִּלְחָהּ ⇐ הַלְבֵּט ⇐ לֹחִיץ קִסֵּם. חוּדְחָהּ חֹבֵשׁ קָרָה

```
Order 0.0 - 1.0 CancelationReason { name, isActive }
```

Operations — פעולות המחלקה

מה המחלקה יודעת לעשות — החלק השלישי בתיבה

התיבה — חלק הפעולות מודגש

Order
- orderId : String - status : OrderStatus - pickupTime : DateTime
+ total() : Money + cancel()

חלק שלישי בתיבה — נכתב בכתב נטוי

פורמט הפעולה

+ method(param : Type) : ReturnType

- שם הפעולה כמו תכונה — camelCase, מתחיל באות קטנה
- פרמטרים עם טיפוסים בסוגריים — (param : Type)
- טיפוס הערך המוחזר אחרי הנקודתיים — Return : (אם קיים)
- נראות בהתחלה — - / # / +
- הפעולות נכתבות בכתב נטוי — כך מבחינים אותן מהתכונות

+ markReady()

פעולה בעלת "תופעת לוואי"

+ cancel()

ללא ערך מוחזר — משנה מצב בלבד

+ total() : Money

מחזירה ערך — טיפוס Money

עיצוב

הפעולות נכתבות בכתב נטוי — כך מבחינים אותן מהתכונות בתיבה.

התרשים הוא תצוגה — לא המודל

מה שלא מופיע בתיבה — לא נעלם. הוא פשוט לא מוצג

פעולה בלי פרמטרים בתרשים — עדיין יש לה פרמטרים

בתרשים מראים מה שרלוונטי לדיון. המפרט המלא — טיפוסים, ברירות מחדל, אילוצים, פרמטרים וערכי החזרה — קיים במודל, גם כשהתיבה לא מציגה אותו. לחצו על מחלקה כדי לראות את המפרט המלא שלה.

Payment

אין לה בכלל תכונות ופעולות?

לחצו לראות 🖱️

MenuItem

```
- name
- price
- available
+ updatePrice(p : Money) : void
+ isAvailable() : Boolean
```

לתכונות אין טיפוסים?

לחצו לראות 🖱️

Order

```
- orderId : String
- pickupTime : DateTime
- status : OrderStatus
+ addItem()
+ cancel()
+ total()
```

לפעולות אין פרמטרים וערך מוחזר?

לחצו לראות 🖱️

מאוחסן או מחושב?

ההחלטה שקובעת אם ערך הוא תכונה או פעולה

מאוחסן ← תכונה

- unitPrice : Money

נשמר בפועל. יש לו מקור — מישהו הזין אותו, או המערכת קבעה אותו ברגע מסוים.

C#: { get; set; } במסד: עמודה

מחושב ← פעולה

+ total() : Money

לא נשמר בשום מקום. מחושב מהנתונים שכבר יש, ברגע שמבקשים אותו.

C#: Total() במסד: אין עמודה

⚠ הטעות הנפוצה

לאחסן ערך שאפשר לחשב — זה מקור אמת שני. מוסיפים שורה להזמנה, שוכחים לעדכן את המונה, והמודל סותר את עצמו.

זו גם טעות אהובה על AI: טיוטות נוטות להוסיף totalAmount ו-itemCount כשדות שמורים.

✓ מתי כן מאחסנים למרות שנראה מחושב

ערך של רגע מסוים — unitPrice בשורת ההזמנה: מחיר בתפריט משתנה, וההזמנה מלפני חודש חייבת להישאר נכונה. גם: ביקורת ורגולציה (הערך מוקפא כפי שהיה), וביצועים על נפחים גדולים — החלטה מודעת, לא שכחה.

השאלה האחת: אפשר לחשב אותו מחדש?

אם אפשר לחשב אותו ממה שכבר שמור, ותמיד לקבל אותה תשובה — אל תאחסנו. הוא פעולה.

אם אי אפשר — הוא עובדה בפני עצמה, ולכן תכונה מאוחסנת.

נריץ את זה על סכום ההזמנה: אפשר לחשב מהשורות ותמיד נקבל אותו סכום ⇒ לא מאחסנים ⇒ **total() : Money** +.

ועל מחיר היחידה בשורה: אי אפשר — המחיר בתפריט השתנה מאז ⇒ מאחסנים ⇒ **unitPrice : Money**.

מה שאפשר לחשב — לא מאחסנים. מה שלא — זו עובדה, ומאחסנים אותה.

Visibility — נראות

מי מורשה לגשת לתכונה או לפעולה של המחלקה

הסימון בתחילת כל חבר (תכונה או פעולה) קובע מי רשאי לגשת אליו — מבפנים, מהיורשים, או מכל מקום.

סימון	נראות	משמעות	דוגמה מהקמפוס
+	Public — ציבורי	כל מחלקה אחרת יכולה לגשת	total() : Money + על הזמנה — כל מי שמשתמש בהזמנה יכול לחשב את הסכום
#	Protected — מוגן	רק המחלקה ותתי-המחלקות (היורשים)	validate() # על תשלום — זמין לתשלום בכרטיס ולתשלום באשראי, לא מבחוץ
-	Private — פרטי	רק המחלקה עצמה	pickupCode : String - על הזמנה — מוסתר מכל מחלקה אחרת

 **הרגל שכדאי לבדוק**
ה-AI נוטה לסמן הכל כ-public. בשלב הניתוח: נתונים רגישים פרטיים (-), פעולות שירות שהמערכת חושפת ציבוריות (+).

בניתוח נסמן נראות בסיסית; העיצוב המלא מתחדד בקוד.

מה אינו מחלקה

מבחן ההתמדה — הפילטר החד ביותר בבחירת המחלקות

האם המערכת שומרת ומנהלת מידע על זה לאורך זמן?

❌ לא — אינו מחלקה

פלט מחושב (נגזר בזמן אמת)

- דוח מכירות · קבלה · אישור הזמנה מחושבים מנתונים קיימים בכל פעם מחדש — אין רשומה עצמאית לנהל.

משתתף שאינו נשמר

- הספק ("הספק מספק סחורה") — מחוץ ל-scope; המערכת לא מנהלת עליו רשומות

✅ כן — זו מחלקה

- הזמנה — נוצרת, נשמרת ומתעדכנת
- פריט תפריט — מנוהל ע"י המנהל, נקרא ע"י הסטודנט
- תשלום — רשומה נשמרת לכל תשלום

המערכת מנהלת עליהן CRUD — לכן הן מחלקות.

⚠️ הטעות מספר 1 של ה-AI בתרשים מחלקות

ה-AI הופך כל דוח / קבלה / אישור למחלקה. הריצו את מבחן ההתמדה — ובדיקת הצלבה: מועמדת שאין לה קשר לאף תרחיש שימוש, כנראה אינה מחלקה.

מחלקה או לא? — חידון

לחצו על כרטיס כדי להפוך אותו. קודם מחליטים בכיתה — אחר כך בודקים

המבחן בכל אחד מהם: האם המערכת שומרת ומנהלת מידע על זה לאורך זמן?



חלוצה → חלוצה' לא מסקט חזש

הקלחת → נהם מיחובש מורומש מובותח חות



אל $\rightarrow$ מולצאן נפסדזש קר חיח

צ → רחם היילא זונח'ל ידע ארומשען

✓ עז

— החייתם חושש, ואולי, מעט! לחזק את צי מא
דיחי ששום. טקייסוא לא חשעל מיביי מיחזח
חחלחח וייזח אוח.

X 4

הוא נ"א. מש"ל לכל חומצות שדחם בשוחם
טלס חז — להחל הוא נ"א חומצ'ל



א' → חספסה ל' צחח חרצויה

צ → חלצו חססו מ, סמו חזקיסל חזאע

✓ עז

ל'אויפט'ל מ'איוואו, ר'פסו מ'לכז, מ'חחזל
 נ'אז ק'ואל מ'לחונאז מ'חזל — ס'וטס מ'זשז

X מזה

אזרח חמשנש'הם שווייץ שיחתה — הלואה וז
השווייץ אל החלטה

X **פְּזוּזָה מְחֻלָּה מֵאֵל**

«פאפארטן געזעמל»-ס — וואו מ'רופט אים.
אין מאדערנע צייטן, איז ער שוין נישט דא.

הצגה מקוצרת של המחלקה

כשהמיקוד הוא הקשרים — מצמצמים את התיבה

כשהדיון הוא על הקשרים בין המחלקות, אין צורך בכל הפרטים. מצמצמים את התיבה ומשאירים רק את מה שחשוב.

Order

שם בלבד

Order

```
- orderId : String
- status : OrderStatus
- pickupTime : DateTime
```

שם + תכונות

Order

```
- orderId : String
- status : OrderStatus
- pickupTime : DateTime
```

```
+ total() : Money
+ cancel()
```

מלא — שם, תכונות ופעולות

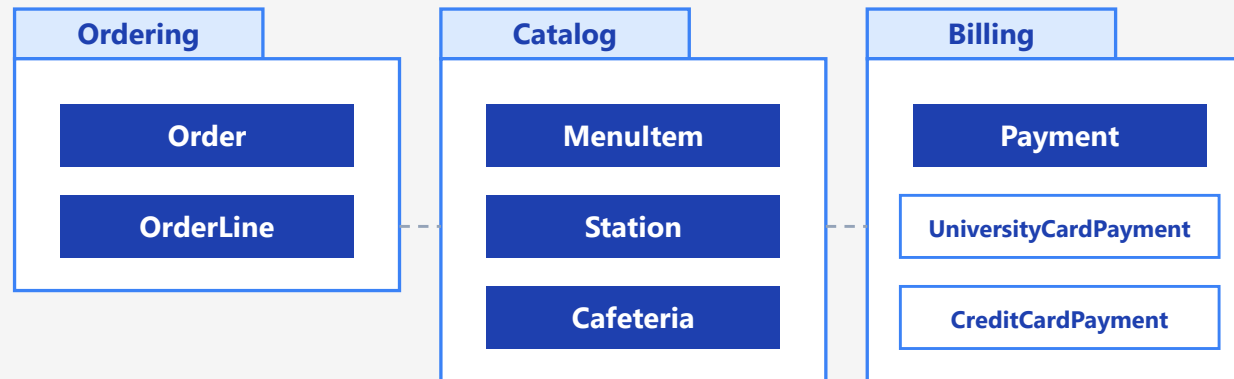
עיצוב

הראו את מה שחשוב לדיון — והשמיטו את השאר כדי למקד בקשרים. המחלקה עדיין מכילה את כל חבריה.

חבילות — Packages

איך שומרים על מודל גדול קריא: מקבצים מחלקות לפי נושא

מודל הקמפוס — מקובץ לשלוש חבילות



הקשרים בין המחלקות נשארים — החבילות רק מארגנות אותן

התיבות בהצגה מקוצרות — שם בלבד

שיוך במרחב שם: `Billing::Payment` · `Ordering::Order`

מה זו חבילה

- יחידת קיבוץ: מחלקות של אותו נושא נכנסות לתיקייה אחת
- מודל של 40 מחלקות הופך לתרשים של 4 חבילות — קודם רואים את המבנה, אחר כך נכנסים פנימה
- מרחב שם: `Ordering::Order` — שם המחלקה ייחודי בתוך החבילה שלה
- שם חבילה נכתב באנגלית, כמו שם מחלקה

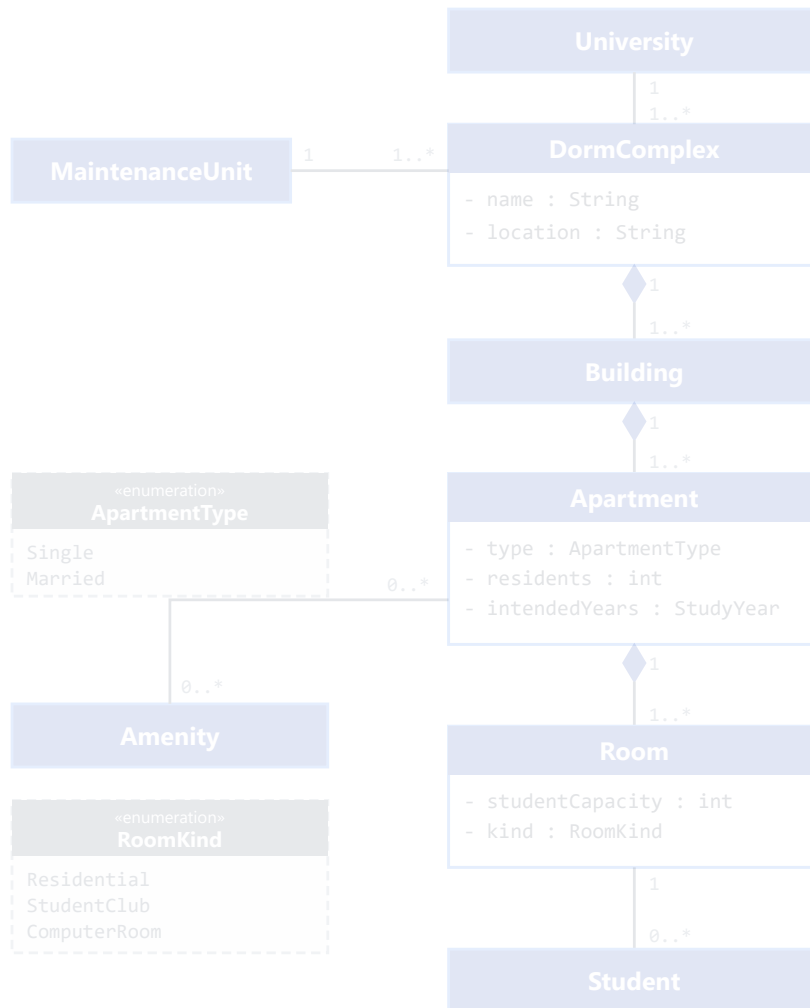
אתם כבר מכירים את זה

ה-`import turtle` מקורס התכנות — חבילה שהמרצה שלכם כתב ואתם ייבאתם. אותו רעיון בדיוק, רק שכאן מציירים אותו.

בקורס הזה לא נשתמש בחבילות — מכירים את המונח, ולא נדרשים אליו בתרגילים.

תרגיל — זיהוי מחלקות

מערכת מידע למעונות: לחצו על כל מונח מודגש — קבלו את ההכרעה, והתרשים ייבנה תוך כדי



התרחיש

ל**אוניברסיטה** כמה **מקבצי מעונות** (מעונות ג', מעונות ד'...), כל מקבץ ב**מיקום** אחר בעיר. לכל המקבצים יש **אב בית** אחד שמטפל בתקלות.

בכל מקבץ יש **בנינים**, ובכל בניין **דירות** — מספר שונה בכל בניין. בחלק מהבנינים יש גם **חדרים שאינם למגורים** (מועדון סטודנטים, חדר מחשבים).

לדירה יש **סוג** (רווקים · נשואים), **כמות דיירים**, ורשימת **תנאים** — דוד חשמל, דוד שמש, כיריים גז, כיריים חשמל... כל דירה מיועדת ל**שנות לימוד** מסוימות.

בכל דירה כמה **חדרי מגורים**, ולכל חדר **כמות סטודנטים** משלו — יש חדרים ליחיד, לשניים וכו'. **סטודנטים** משובצים לחדרים על ידי משרד המעונות.

הערה: כמות הסטודנטים בחדר נקבעת על ידי **דיקאנט הסטודנטים**. היא קבועה בדרך כלל — אבל הדיקאנט יכול לשנות אותה (כך הפכו חדרי זוגות לחדרי יחיד).

■ מחלקה ■ תכונה ■ רשימת ערכים ■ מלכודת / לא מחלקה

מפת הקשרים

חמישה סוגי הקשר שבהם נשתמש — ומתחתם: ארבע הדרגות של Association



תלות
Dependency

קו מקווקו + חץ פתוח למערכת חיצונית שאנחנו קוראים לה.

«external system»



הכללה / הורשה
Generalization

➤ יחס is-a. רק כשונים בקשרים — לא בתכונות.



הרכבה
Composition

◆ החלק לא קיים בלי השלם.



צבירה
Aggregation

◊ שלם-חלק. החלק קיים גם בלי השלם. לזיהוי בלבד — לא מייצרים.



אסוציאציה
Association

קו מלא, בלי חץ — הקשר הבסיסי. קרדינליות בשני הקצוות.

כולן דרגות של קשר Association — מהפשוט למורכב

מתי הקשר הרגיל (הכרטיס הראשון למעלה) לא מספיק, ומה עושים אז

קשר n-ary



מחלקה מתווכת



מחלקת קשר



קשר רגיל

בחרו תמיד את הדרגה הפשוטה ביותר שמספיקה. קשר n-ary — לפרק למחלקה מתווכת. שימו לב: מחלקה מתווכת אינה סימון על הקשר — היא מחליפה אותו בשתי אסוציאציות ומחלקה באמצע.

עיצוב

בשקפים הבאים נצלול לכל קשר בנפרד — על מודל הזמנת האוכל בקמפוס.

Association + קרדינליות

הקו שמחבר שתי מחלקות — וכמה מכל צד

הכללים

- קו מלא — בלי חץ (חץ מכון לא בשימוש על מודל התחום)
- קרדינליות בשני הקצוות — תמיד
- המשמעויות:
 - 1 = בדיוק אחד
 - 0..1 = אפס או אחד (אופציונלי)
 - 1..* = אחד או יותר
 - 0..* = אפס או יותר (כל כמות)

דוגמה מהקמפוס

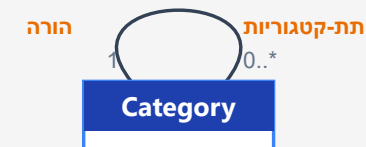


סטודנט אחד מבצע אפס או יותר הזמנות; כל הזמנה שייכת לסטודנט אחד בדיוק.
קו מלא, בלי חץ.

אילוץ {or}

הזמנה משויכת לתשלום בכרטיס אוניברסיטה {or} תשלום באשראי — אחד מהם, לא שניהם. מסמנים {or} על שני הקווים היוצאים.

קשר רקורסיבי (מחלקה לעצמה)



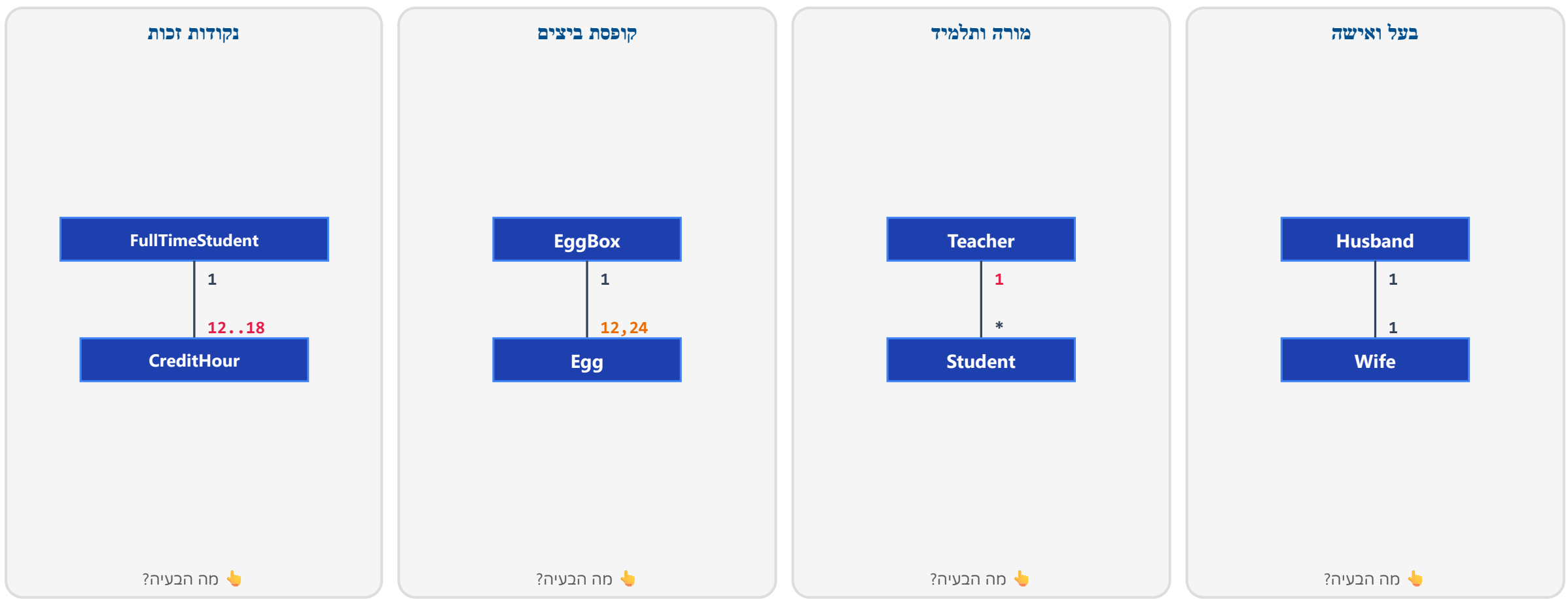
לקטגוריה יש הורה אחד ותת-קטגוריות משלה — אותה מחלקה בשני הקצוות.
שמות תפקיד על הקצוות הם אופציונליים — כאן בלי הם לא היינו יודעים מי ההורה.

⚠️ הטעות הנפוצה של ה-AI

משאיר קצה אחד ריק, או מנחש 1 בלי לבדוק. תמיד לוודא קרדינליות בשני הקצוות — ולשאול מול המקור: כמה מכל צד באמת?

הקרדינליות הקלאסית — ולמה רובה שגויה

לחיצה ראשונה — מה לא בסדר. לחיצה שנייה — מודל טוב



שלוש שאלות: 1. תמיד? או רק ברוב המקרים — ומה קורה במיעוט? 2. גם עם היסטוריה? ה-1 נכון היום, ולא לאורך זמן 3. לפי איזה חוק — ומי רשאי לשנות אותו?

קרדינליות היא החלטה עסקית — ולפעמים המספר שגוי כי המחלקה שגויה.

Unified Modeling Language = UML · שפת מידול אחידה · Artificial Intelligence = AI · בינה מלאכותית

איזו קרדינליות כותבים — ומה כבר שייך למימוש

אנחנו בשלב העיצוב. התרשים שלכם הוא המפרט — והמימוש נגזר ממנו

✗ מספרים ספציפיים — אלה לא קרדינליות

12,24 12..18 1..20 2

אף שפה ואף מסד נתונים לא אוכפים אותם, והם מקבעים חוק שמשתנה. מקומם באילוץ בפתק ובנתון — לא על הקו.

✓ ארבע צורות — וזה הכול

0..* 1..* 0..1 1

כולן תשובות לשתי שאלות בלבד: האם זה יכול להיות אפס? והאם יכולים להיות יותר מאחד?

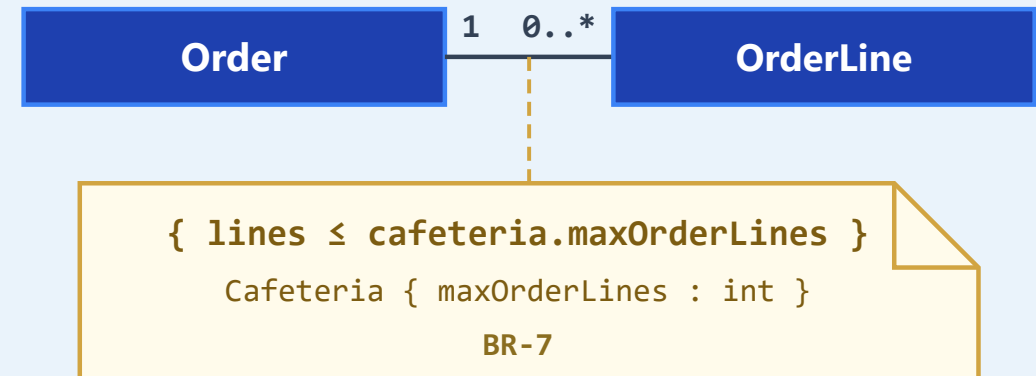
Implementation — תרשים מימוש
מה שנגזר מזה בקוד



במימוש	בעיצוב
List<OrderLine>	0..*
שדה חובה, לא ניתן ליצור בלעדיו	1
canAddLine()	{ }

הקשר נעלם והופך לשדות, הריבוי הופך לList, והאילוץ הופך לפעולה. כאן מותר מחלקה כתכונה — בעיצוב לא.

Design — עיצוב
מה שאתם מייצרים ומסרים



איפה כותבים את החוק? בפתק (Note) שמחובר לקו הקשר, ובתוכו האילוץ בסוגריים מסולסלים. והחוק עצמו כבר קיים בקטלוג מהרצאה 3 — לכן מפנים אליו (BR-7) במקום לשכתב אותו.

עיצוב

אתם מייצרים את תרשים העיצוב; ה-AI גוזר ממנו את המימוש — מה שלא אמרתם, הוא ימציא.

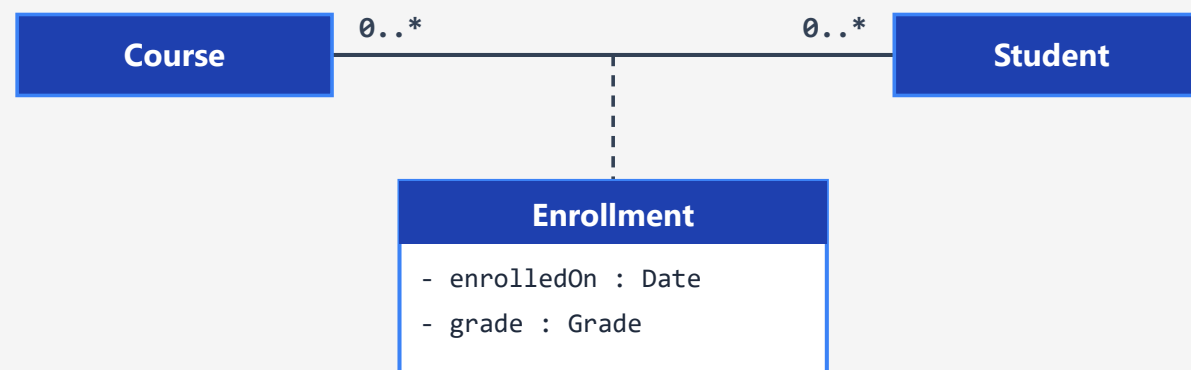
מחלקת קשר — כשלקשר עצמו יש נתונים

הציון אינו של הסטודנט ואינו של הקורס — הוא של הקשר ביניהם

איפה שמים את הציון?

× על הקורס? יש בו ציון אחר לכל סטודנט

× על הסטודנט? יש לו ציון אחר בכל קורס



✓ הציון קיים רק בזכות הצירוף של השניים — ולכן הוא נתלה על הקשר עצמו. המחלקה שנתלית כך נקראת מחלקת קשר.

איך מציירים

1 הקשר נשאר על מקומו — לא מוחקים אותו ולא מפצלים אותו.

2 קו מקווקו יוצא מאמצע קו הקשר אל קופסת המחלקה. בלי חץ ובלי יהלום.

3 הקרדינליות נשארת על קו הקשר; על הקו המקווקו אין קרדינליות — הוא לא קשר.

4 הקופסה היא מחלקה רגילה לכל דבר: שלושה מדורים, שם UpperCamelCase-2.

5 השם הוא שם עצם של הקשר — Enrollment, Membership, Employment — ולא פועל ולא "קשר".

1 למי שייך הנתון?

לא לצד אחד ולא לשני. "של מי הציון?" — של שניהם יחד. אם התשובה היא צד אחד, זו פשוט תכונה שלו.

2 כמה מופעים לכל צמד?

אחד לכל היותר. סטודנט אחד + קורס אחד = הרשמה אחת. הצמד הוא שמזהה את המופע — אין לו מזהה משלו.

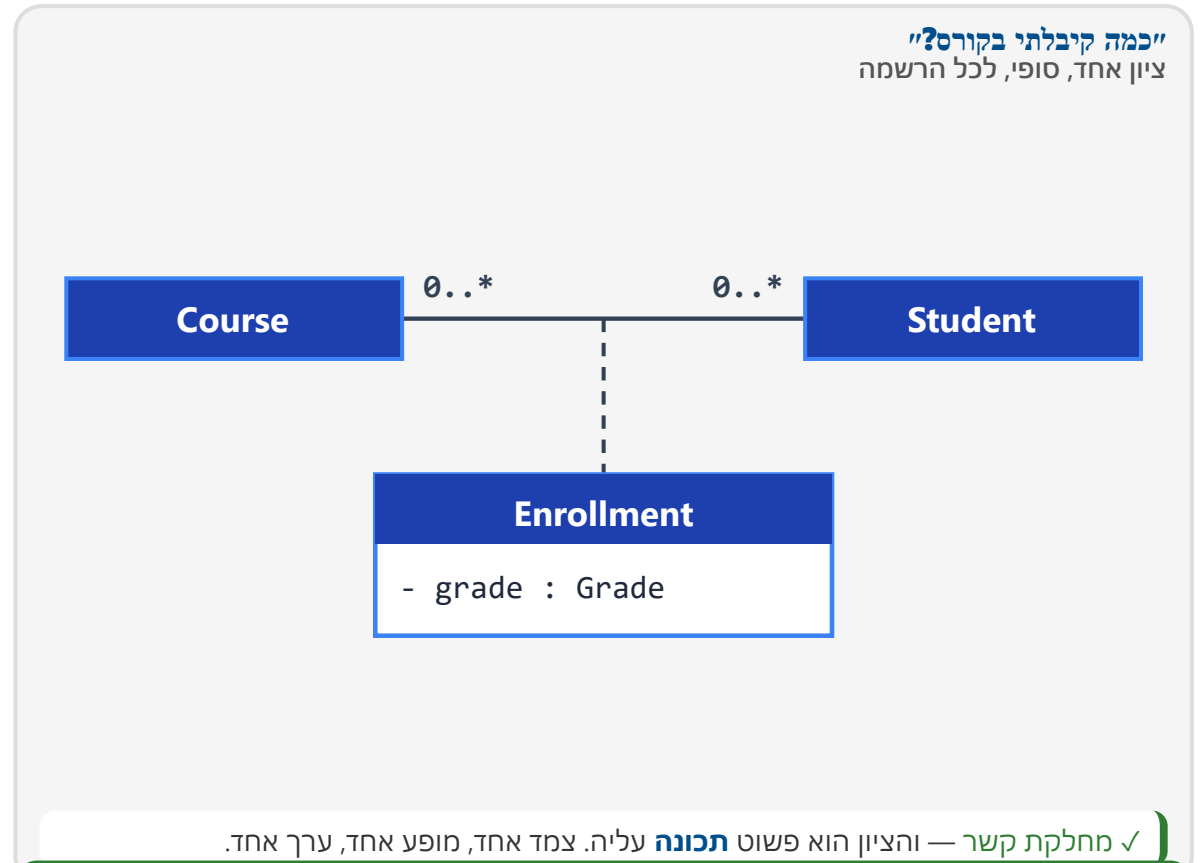
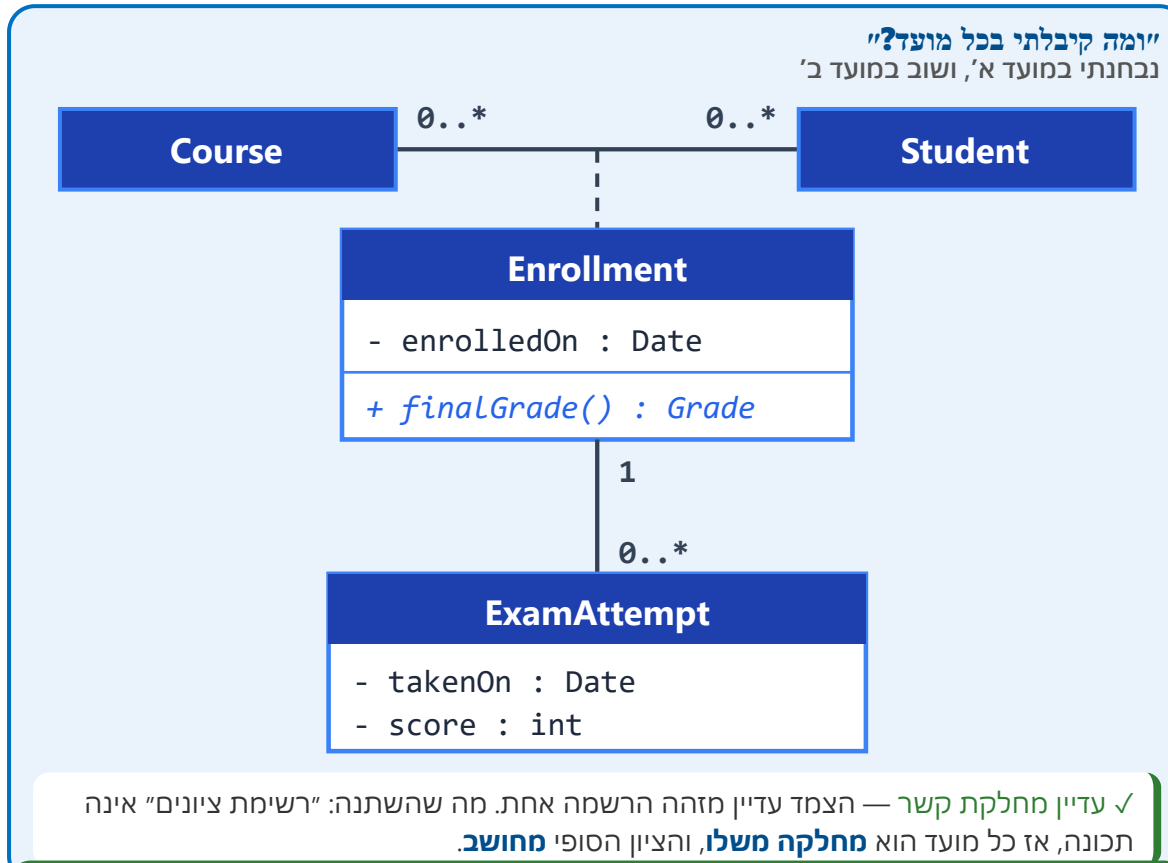
3 יש לו חיים בלי הקשר?

אין. הוא נולד עם הקשר ומת איתו. אף אחד בעסק לא אומר "הרשמה 4471" — אומרים "ההרשמה של דנה לקורס".

ובעיקר מבחן 2: צמד שיכול להופיע פעמיים — כבר אינו מחלקת קשר.

מחלקת קשר היא מחלקה לכל דבר

אותה דוגמה, שתי שאלות: "כמה קיבלתי בקורס?" מול "מה קיבלתי בכל מועד?"



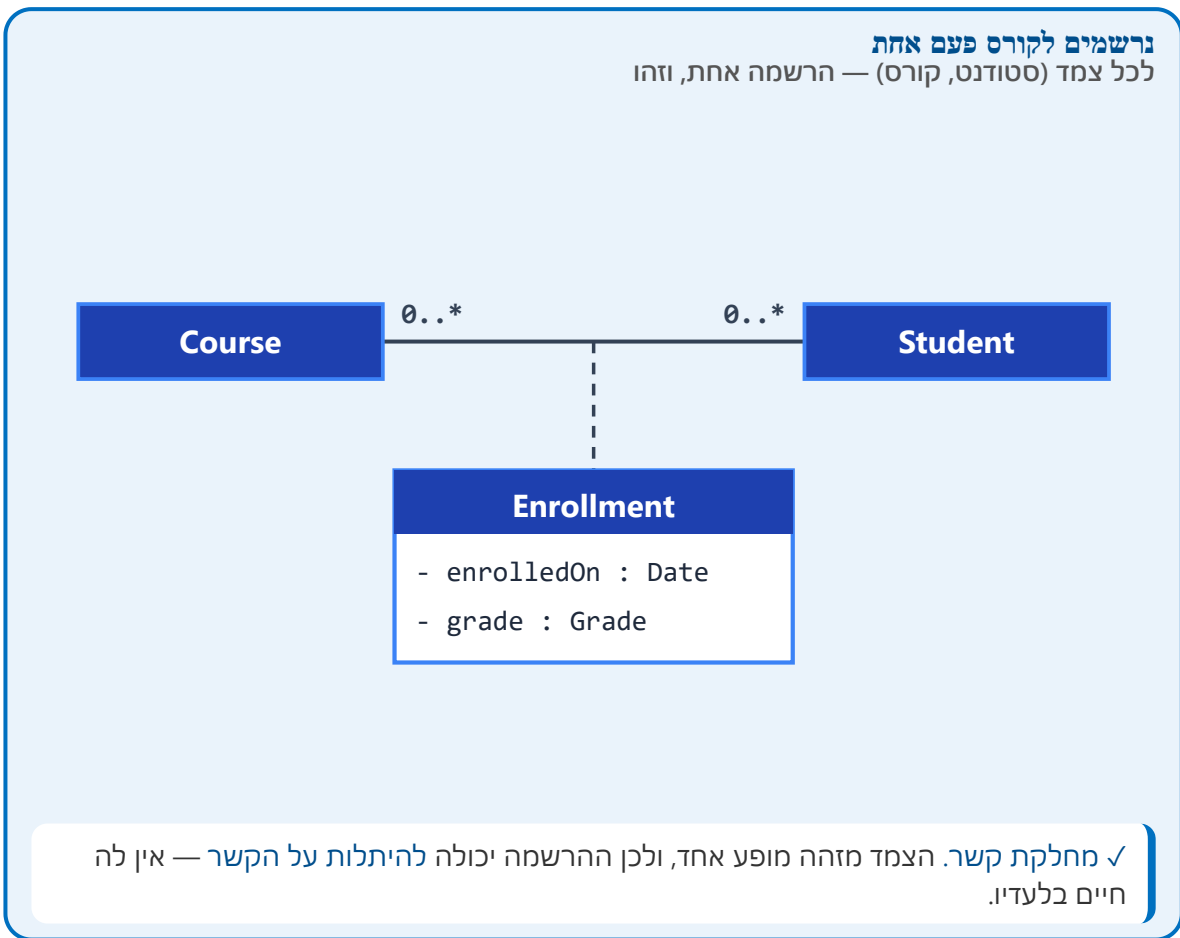
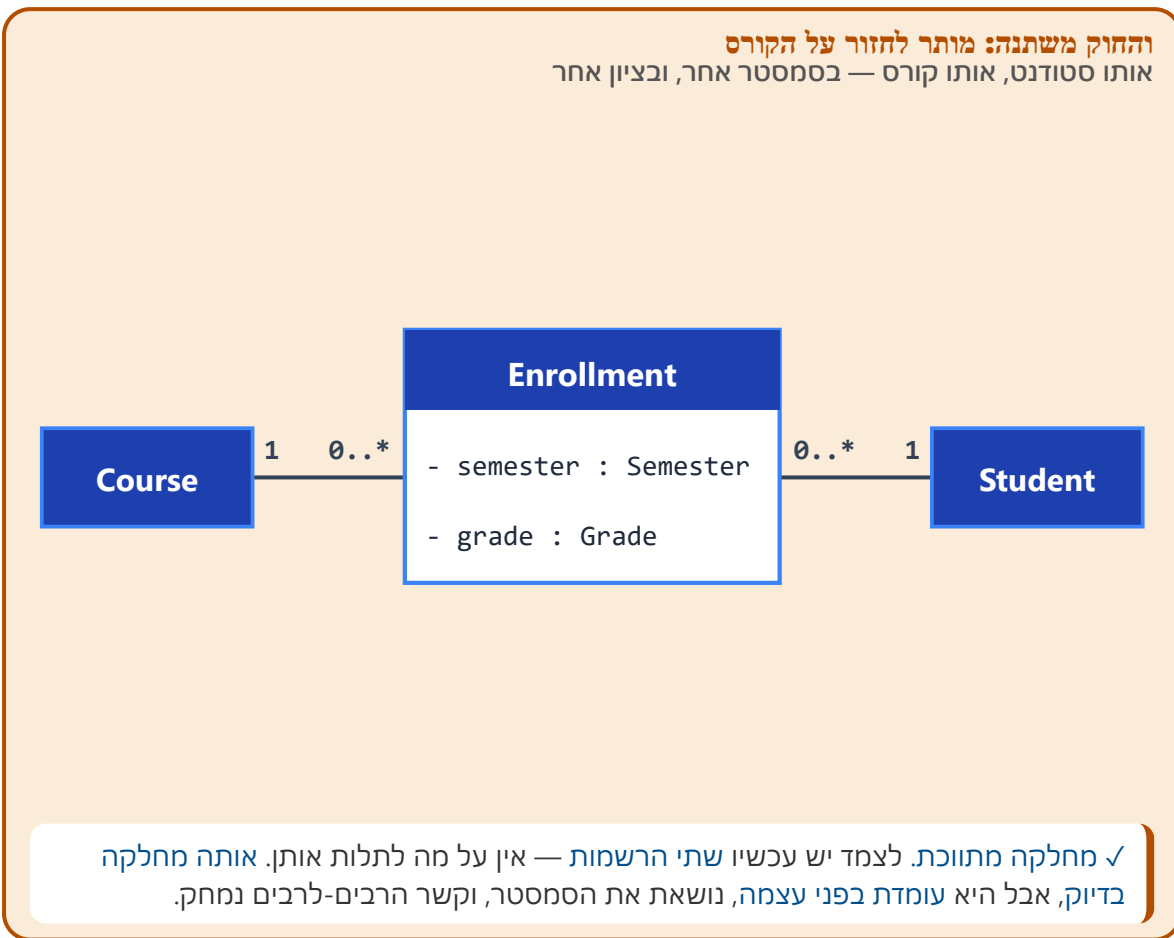
✓ **מותרים לה קשרים**
ExamAttempt תלוי **בהרשמה** — לא בסטודנט (באיזה קורס?) ולא בקורס (של מי?). רק להרשמה יש את שני הצדדים, ולכן היא הצד הנכון לקשר. **מה שעדיין אסור**: שני מופעים לאותו צמד.

✓ **מותרות לה פעולות**
אין ב-UML שום כלל שאוסר זאת — מחלקת קשר היא גם **מחלקה** וגם **קשר**, ולכן יש לה תכונות, פעולות והורשה. נדמה שאין, כי במודל תחום כמעט לא מציירים פעולות בכלל. וכאן חל בדיוק אותו כלל: **מחושב** ← פעולה.

מחלקת קשר אינה "חצי מחלקה" — מה שאין לה זהות ומחזור חיים משלה, וזה הכול.

מתי מחלקת קשר מספיקה — ומתי כבר לא

אותו סטודנט, אותו קורס, אותו ציון. מה שמכריע הוא חוק עסקי — האם אפשר להירשם פעמיים



ועוד שני סימנים שהמופע הוא "דבר" ולא "קשר": יש לו מצב — הרשמה שבוטלה, שורה שהוחזרה · ומישהו מזכיר אותו בשמו — "ההרשמה מסמסטר ב'". לקשר אין מצבים, ואף אחד לא מדבר עליו בפני עצמו.

שאלה אחת מכריעה: האם הצמד מזהה את המופע? כן ⇐ מחלקת קשר · לא ⇐ מחלקה מתווכת.

דוגמה נוספת — שורת הזמנה בקמפוס

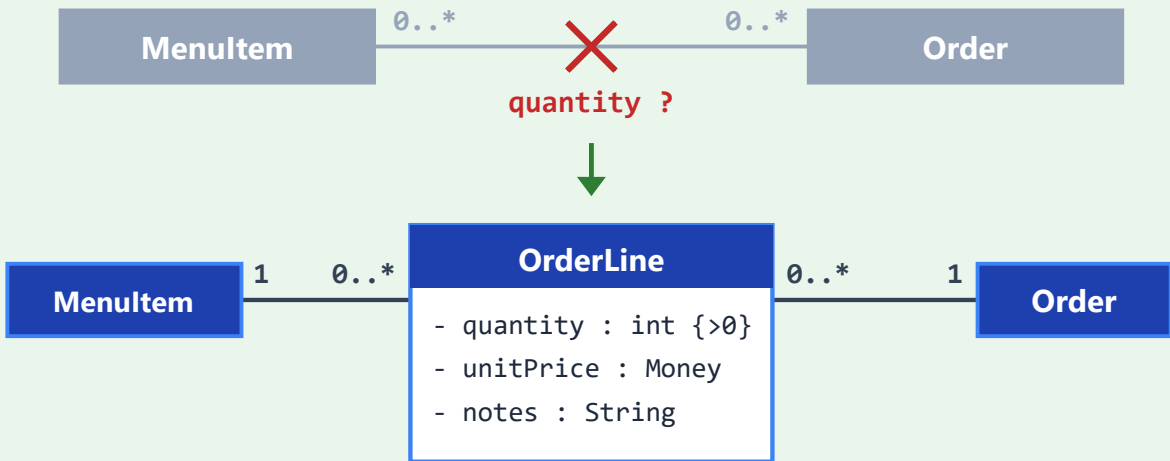
שניצל עם בצל ושניצל בלי, באותה הזמנה: אותו צמד — שתי שורות

אותה הכרעה, על המודל שלנו

- הצמד (הזמנה, פריט תפריט) חוזר: שתי שורות לאותו פריט, עם כמויות והערות שונות. אין על מה לתלות אותן — מחלקת קשר מחזיקה מופע אחד לכל צמד.
- ולשורה יש חיים משלה: אפשר לבטל שורה אחת בלי לבטל את ההזמנה, ולשנות בה כמות.
- ולכן OrderLine היא מחלקה מתווכת — היא מחליפה את קשר הרבים-לרבים בשתי אסוציאציות רגילות, ונושאת את מה שנולד מהקשר.

ואילו העסק היה אומר אחרת: "כל פריט מופיע בהזמנה פעם אחת, עם כמות" — אז הצמד היה מזהה, וזו הייתה מחלקת קשר. וכך בדיוק בנוי OrderItem בפרויקט לדוגמה של הקורס. אותה צורה בקוד — החלטת זהות אחרת.

מה שמצויר במודל הקמפוס

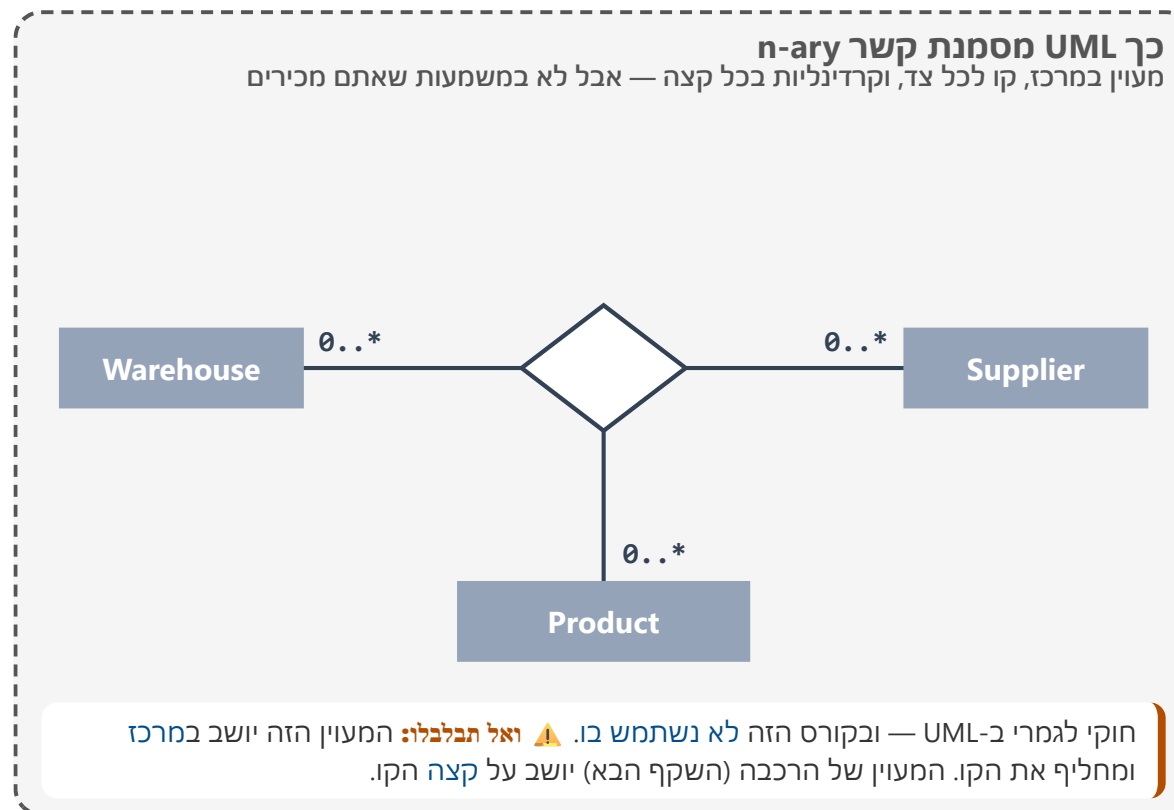
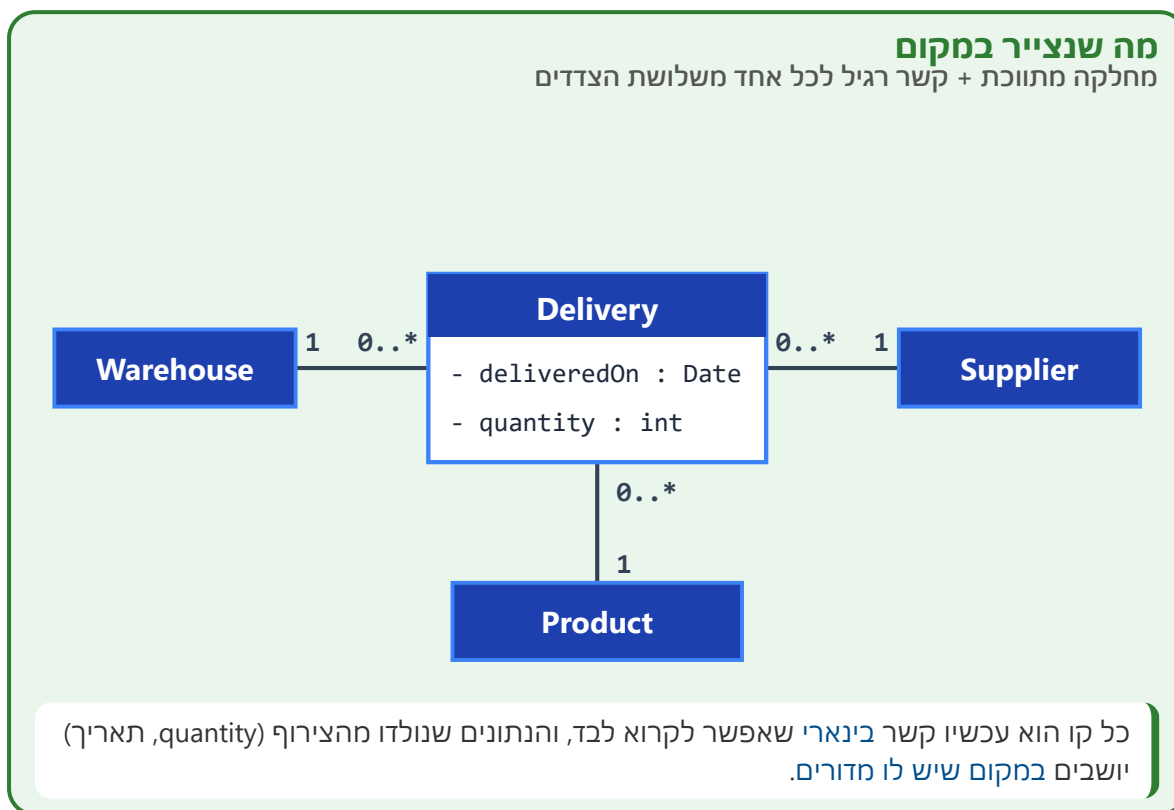


קשר הרבים-לרבים המקורי נמחק, ובמקומו שתי אסוציאציות רגילות ומחלקה באמצע. שימו לב: OrderLine אינה תלויה על קו — היא מחליפה אותו.

היבט	מחלקת קשר (Association Class)	מחלקה מתווכת (Mediating Class)
מבחן ההכרעה	הצמד מזהה מופע אחד לכל היותר	הצמד יכול לחזור, או שלמופע יש חיים משלו
סימון	הקשר נשאר; קו מקווקו אל אמצע הקו	מחליפה את הקשר בשתי אסוציאציות
במסד הנתונים	טבלה שמפתחה הוא הצמד (מפתח מורכב)	טבלה עם מפתח משלה, כמו כל מחלקה

קשר בין שלושה — הסימון שקיים, ולא נשתמש בו

”ספק X סיפק מוצר Y למחסן Z” — עובדה אחת שדורשת שלושה צדדים יחד



❌ אף אחד ב-**downstream** לא מייצג את זה
אין לזה מבנה בקוד ואין טבלה כזאת במסד. בסוף תמיד נוצרת טבלה עם מחלקה באמצע — אז עדיף לצייר אותה מלכתחילה.

❌ אין איפה לשמור נתונים
הכמות והתאריך שייכים לשלושת הצדדים יחד — ולמעוין אין מדורים. כדי לשמור אותם צריך ממילא מחלקה.

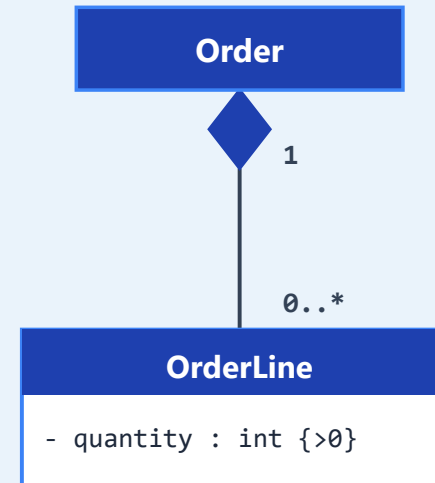
❌ הקרדינליות אומרת משהו אחר לגמרי
שימו לב שהמספרים בשני התרשימים שונים — וזו לא טעות: במעוין, 0..* בקצה של מוצר נספר לכל צירוף של ספק ומחסן. בפירוק, 1 אומר פשוט: כל אספקה היא של מוצר אחד. שתי שאלות שונות — ורק השנייה נקראת.

שלושה צדדים ומעלה ⇐ תמיד מחלקה מתווכת, ומכל צד קשר רגיל. את המעוין משאירים בתקן.

הרכבה • — החלק חי ומת עם השלם

המעוין המלא יושב על השלם, והוא אומר משהו עם תוצאות — לא סתם "שייך ל..."

מה הסימון אומר



- 1 תלות קיומית: נמחקת ההזמנה — נמחקות איתה כל שורותיה. אין "שורה יתומה".
- 2 בעלים אחד בלבד: אי אפשר שאותה שורה תשתייך לשתי הזמנות. שיתוף שולל הרכבה — וזה המבחן שנופלים בו יותר.
- 3 ומה שמחייב את היצירה הוא המספר: ה-1 אומר ששורה לא נוצרת בלי הזמנה; ה-0..* אומר שהזמנה כן יכולה להיוולד ריקה.

מתי כן, ומתי בכלל לא

- ✓ הזמנה ♦ שורת הזמנה — לשורה אין משמעות מחוץ להזמנה שלה
- ✓ בניין ♦ חדר — חדר לא "עובר" לבניין אחר
- ✓ חשבונית ♦ שורת חשבונית — אותו דפוס בדיוק
- ✗ הזמנה ♦ פריט תפריט — הפריט משותף להזמנות רבות וקיים בלעדיו. זו אסוציאציה רגילה
- ✗ מחלקה ♦ עובד — סוגרים מחלקה, העובד עובר. לא נמחק
- ✗ קפיטריה ♦ עמדה — העמדה שורדת ועוברת. זו כבר צבירה — השקף הבא

בעיצוב — לא אופציונלי

המספרים כבר אמרו שלשורה תמיד יש הזמנה. מה ש-♦ מוסיף הוא מה קורה במחיקה — השורות הולכות עם ההזמנה. בלי היהלום זה נשאר לא מוגדר.

במימוש — יש לזה תוצאות

מפתח זר NOT NULL + מחיקה מדורגת (ON DELETE CASCADE) · החלק נוצר דרך השלם ואין לו מסך או CRUD משלו · ואין אליו הפניה שנייה.

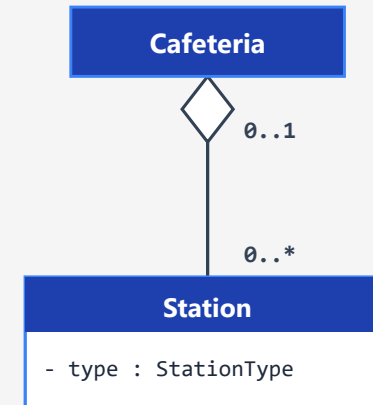
♦ הוא הוראה למי שמממש — ולכן מה שסומן בו בטעות עלול להימחק בטעות.

Unified Modeling Language = UML · שפת מידול אחידה · Create, Read, Update, Delete = CRUD · יצירה, קריאה, עדכון, מחיקה · Artificial Intelligence = AI · בינה מלאכותית

צבירה ♦ — מזהים, לא מייצרים

אותו "שלם-חלק", אבל בלי תלות קיומית — ובלי שום תוצאה במימוש

מה הסימון אומר



העמדה שייכת לקפיטריה — אבל חיה בלעדיה. נסגרת הקפיטריה, העמדה עוברת למקום אחר.
ומה שמתיר או אוסר יצירה הוא המספר, לא היהלום: $0..1 \Leftarrow$ אפשר לייצר עמדה לפני שיש קפיטריה $\cdot 0..* \Leftarrow$ קפיטריה יכולה להיפתח בלי אף עמדה.

איפה תפגשו אותו — ואיפה זו סתם טעות

- ✓ קפיטריה ♦ עמדה — העמדה מיטלטלת ושורדת את הסגירה
- ✓ צוות ♦ עובד · מועדון ♦ חבר — חברות, לא בעלות
- ✗ סתם "יש לו" — כל אסוציאציה היא "יש לו" במובן כלשהו. ♦ אינו תחליף לשאלה מה באמת הקשר
- ✗ כשהחלק נמחק עם השלם — זו הרכבה ♦, לא צבירה
- ✗ כשזה בסך הכול קטלוג — תפריט "מכיל" פריטים, אבל זו רשימה, לא שלם-חלק

במימוש — שום דבר

אין מילת מפתח בשום שפה · במסד: מפתח זר רגיל, שמותר שיהיה ריק · מחיקת השלם לא נוגעת בחלק · בקוד: אותו `List<Station>` בדיוק כמו באסוציאציה רגילה.

בעיצוב — כן אופציונלי

התקן עצמו אומר שהמשמעות המדויקת של צבירה משתנה בין תחומים ובין ממדלים. קו רגיל אומר למערכת בדיוק אותו דבר: כל מה שבאמת מחייב יושב במספרים. ♦ מבטא כוונה — לא חוק.

הכלל בקורס: אנחנו לא מייצרים ♦ בכלל — לא בתרגילים, לא בפרויקט ולא בכלי. במקומו: קו רגיל. לזהות ולהבין ♦ בתרשים של מישהו אחר: כן — וזו הסיבה שהשקף הזה קיים. וגם: AI מפזר ♦ בנדיבות; זו לא שגיאה, זה רעש.

♦ משנה את המימוש · לא משנה כלום. לכן על ההרכבה חושבים, ועל הצבירה לא מתעכבים.

איזה קשר? — חידון

לחצו על כרטיס כדי להפוך אותו. קודם מחליטים בכיתה — אחר כך בודקים

שתי שאלות בלבד: מה קורה לחלק כשמוחקים את השלם? והאם מישהו נוסף מחזיק בו? $\Leftarrow$ שתי תשובות שורדות לתרשים: $\blacklozenge$ הרכבה או קו רגיל.

 הצגה
מיושם מיושם יום
 — סרטון יומן מיו, אחרי יומן "מיושם" אל חזית
 חזית יום.
 819219 — 9876543210

 **הצגה**
מיושם ממוחשב יחיד
 — סרטון וינטגס, מחא וינטגס "מחשבים" אל חרז
 חרז יחיד.
 Building —  אמצעים

 **הצגה**
מיוזם ממומנת יזם
 — סרטון ייחודי מאו, אחא וייטל "מחשבות" אל חזית
 חזית יזם.
 Building —  אמצעים

רָחֵם אֱלֹהֵינוּ
לְחַיֵּי חַיֵּי הַחַיִּים
— רַחֵם וְאֵל שׁוֹמֵר חַיֵּי חַיֵּי הַחַיִּים —
רַחֵם וְאֵל שׁוֹמֵר חַיֵּי חַיֵּי הַחַיִּים

רָחֵם אֱלֹהֵינוּ
לְחֵם-בָּלֶם אֵל עַל-כָּל
— רָשָׁק וְאֵץ שֵׂי. וְיִשְׁעוּ לָשׁ "לְחֵם" וְיִזְיֵא מִחֵם זָחֵא מֵא
רָשָׁק חֲקֻלָּחֵם, לְוִיץ וְיִלָּשׁ חֻמְשָׁל קִירֵץ מֵאוּ

תַּחֲבֹתָא — סַטְרֵא

רָחֵם אֱלֹהֵינוּ
לְחַיֵּינוּ—מֵלֵךְ אֱלֹהֵינוּ

— רַחֵם וְאֶסֶף שׁוֹמְרֵי חֻמְרוֹתֵינוּ וְיִשְׁלַח מַלְאָכָיו לְרַחֵם אֶת כָּל הַיְּהוּדִים וְאֶת כָּל הַיִּשְׂרָאֵלִים.

תְּהִי שְׁמִי כְּשֵׁם אֱלֹהֵינוּ — אֱמֵן

ליגה זק ⇒ זימזאם קלח-פלא
מינחה ימצא לשון, חכמה שמש

רחא בצר רבון וסחר חוש, קחתו לא
אז — משה יל סייק קלה מא או חכה
ויביח חלש וד. ביכלא יל חיכות איח חיכות.

Car — Wheel

ליגה זק ⇒ זימזאם קלח-פלא
מינחה ימצא לשון, חכמה שמש

רחא בצר רבון וסחח חוש, קחתו לא
אז — פלשה יל סייק קלה מא אוה וחכה
ויביח חלש וד. ביכלא יל חיכוח איח חיכוח.

Car — Wheel

זיגט זיך $\Rightarrow$ זיפזאגט קלאר-פלאר
 מינאמאל יאמאל פאראן, מאלאמאל פאראן
 און אפאמאל פאראן. ⚠️
 און אלס — פאראן יאמאל מיין קלארע און און
 יאמאל פאראן און פאראן און פאראן.

Car — Wheel

הצגה
פיצוץ פיצחיה יזש
 זה חייב להיות לזכור, חפץ פוחם מייך אל דומה
 חפץ חפץ.

Book — Page


הצגה
פיצוץ פיצחיה יזש
 זה חייב להיות לזכור, חפץ פוחם מייך אל דומה
 חפץ חפץ.

Book —  Page

◆ **הכנה**
פיצוץ פינחמה יזש
מז חויחל לוכי וזיאו, חפסל פוחם מייך אל דומש
חחא חפס.

Book — ◆ Page

לִיגָזָה זָה
 "וְלֹא שִׁי": חֲזוֹנָא

בְּזֶשׁ אֶשְׂרֵי לֵבָא — "פִּימָשׁוּר שִׁי פִּילֹחֵה חִיבָלִי"
 חֲרִיטָס דְּרוּשׁ יִאֲדוּבֹו חוּמָקָם חֲבַב

Isitiqzoh — 7070707

לִיגַת זָה
"וְזֶה שֵׁי": חֲזוּכִים

בְּשֵׁי אֶסְרָא לֹא — "פִּימֻשׁוֹ שֵׁי פִילֻחָהּ חִיכֵל"
חֲרִיסָה דְרוּשׁ וְיִצְחוּבֵי חוֹמוֹקִים חֲדָב

Isitqzoh — rctotod

לִיגַח זָק
 "וְלֹא שִׁי": חֲזוֹלָה

כְּדֵּשׁ אֶשׁוּר לֹאֵא — "פִּי־אֶשׁוּר שִׁי פִּלְחוּחַ חִיבֵלִי"
 חֲרִיטֵס דְּרוֹשׁ יִאֲדוּבוּ חוֹמֻקֻס חֲבֵב

Isitiq20H — 7070707

◆ **השקפה**
מימוש פיתוחות יזם

הזדמנות היא מיטבם; מיוסם הזדמנות שחלף קיים
— וחיוב קיומו קיים.

Constrains —◆ Constraints

◆ **השקפה**
מימוש פיתוחות יזם

הזדמנות היא מיטבם; מיוסם הזדמנות שחלם קישם
— וחיזק קידום קישם.

Constrata —◆ Class

◆ **השקפה**
מימוש פיתוחות יזם

הזדמנות היא מיטבם; מיוסם הזדמנות שחלם קישם
— וחיזק קידום קישם.

Constrata —◆ Class

אולטרא סאונד: איינפירטאנציע
פאר אונזערע קינדער

אולטרא סאונד: איינפירטאנציע
פאר אונזערע קינדער

אולטרא סאונד: איינפירטאנץ פאר
אונזערע קינדער

לִיגָה זָה ⇒ זִישׁוּמַשׁ הָלֵח-פֶּלֶשׁ
הַבְּרָחָה אֵל הָאֵל לִכְסוֹ — יַחֲיוּסָא בִּיכִרָה

הָאֵל לִכְסוֹ — חֻסְיָו יֵלֵךְ סוֹטְמַחֲמָה קֹלָח אֹחֻז שׁוֹמֵר
יַחֲיוּסָא סוֹטְמָה לֵאל בְּרָחוּמָה, רַחֲמוֹהֶם, דְּרַשְׁמֵי קוֹשִׁישׁ חֲסָדִים

אֲנִיגְמָה — פֶּלֶשׁ וְהִנֵּה

לִיגָה זָה ⇒ זִישׁוּמַשׁ הָדָח-פְּלֶשֶׁת
הַבְּרָחָה אֵל תְּהֵא לֹכְסו — יַחֲיוֹנָא בִּיטָר

תְּהֵא לֹכְסו — חֻסְיֹן וְיֹלֵךְ סוֹטְמַחֵם קֹלָח אוֹחַ שׂוֹמֵם
 יַחֲיוֹנָא סוֹטְמֵם לֹא עֲבָדוֹמוֹ, רַחֲחוּם, דְּעַמְשָׁן פְּוִשִׁישׁ מְשׁוֹץ

אֶתְהִיבְרַחֲלָה — תְּהִיגְנֵה

לִיגָה זָה ⇒ זִישׁוּמַשׁ הָלֵךְ-פֶּלֶשׁ
הַבְּרָחָה אֵל הָאֵל לִכְבוֹד — יִחְיֶה אֵל הַיָּמִים

הָאֵל לִכְבוֹד — חֻסְיוֹ יֵלֵךְ סוֹטָמָה הָלֵךְ אֹהֶל שׁוֹמֵר
יִחְיֶה סוֹטָמָה לֵאלֹהֵי הַבְּרָחָה, הַחֲחֹמֶה, הַדְּמַשׁק וְשִׁישׁ הַבָּשָׂה

אֵל הַיָּמִים — הַיָּמִים

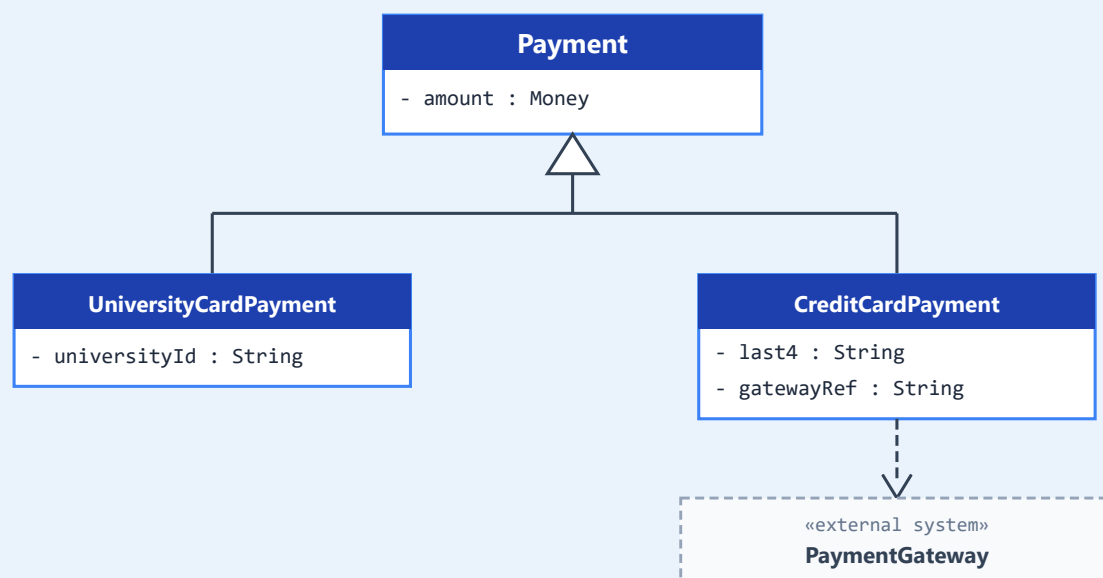
עיצוב

מתוך שלוש האפשרויות — רק שתיים מגיעות לתרשים: ♦ כששני המבחנים עוברים, ואחרת קו רגיל.

הורשה — ומתי היא מוצדקת

הסימון קל: משולש חלול ▷ אצל ההורה, הקו מהילד אליו. השאלה הקשה היא מתי

ולמה במודל שלנו זו כן הורשה



ההצדקה אינה "לכל אחד יש שדה אחר" אלא **הקשר**: רק תשלום באשראי פונה לשער חיצוני. ובנוסף התכונות **סותרות** — last4 ו- universityId לעולם לא יחיו יחד.

✗ נשמע כמו סיבה — ואינו

תכונה שיכולה להיות ריקה. תכונה בלי ערך אינה בעיה במחלקה. במסד נתונים עמודה ריקה ברוב השורות נראית כמו בזבז — ומשם מגיעה האינטואיציה השגויה.

פעולה שונה. בשלב ה**ניתוח** כמעט לא מציירים פעולות בכלל. (בקוד, פולימורפיזם כן שיקול — הרצאה 13).

תווית, תפקיד או סטטוס. $\Leftarrow$ enum או תכונה.

✓ שתי סיבות אמיתיות (ומבחן אחד)

הקשרים שונים. תת-סוג אחד מחובר למחלקה שהשני אינו מחובר אליה. אין דרך לומר את זה בלי תת-מחלקה.

הטבע שונה — ובודקים לפי **תכונות שסותרות זו את זו**. לא "יש לו עוד שדה" אלא או־או: מי שמשלם באשראי לעולם לא יחזיק מזהה אוניברסיטאי.

המבחן המכריע: האם מופע יכול **לעבור** מסוג לסוג? כן $\Rightarrow$ זו אינה הורשה — אובייקט לא מחליף מחלקה.

⚠ **המלכודת שחוזרת כל שנה** — מחלקה לכל תפקיד: Employee ▷ מנהל / קופאי / טבח. כמעט תמיד זו **תכונה** או **enum**, לא הורשה — כי התפקיד **משתנה** (קופאי מקודם למשמרת), וכדי "להחליף מחלקה" צריך למחוק את האובייקט וליצור אותו מחדש. רק אם לתפקיד יש **קשרים משלו** — למשל מנהל מאשר הזמנות ולקופאי אין קשר כזה — יש על מה לדבר.

עיצוב

תכונה ריקה ופעולה שונה — לא מספיקות. קשרים שונים, או תכונות שסותרות זו את זו — זו הורשה.

איזה קשר — אם בכלל?

לחיצה על מקרה — שתי דרכים למדל אותו. מחליטים בכיתה, ואז לחיצה נוספת — תשובה ושיקולים

1



הזמנה ושורת הזמנה — האחת בלי השנייה?



2



קפיטריה והעמדות (חם/קר) שבתוכה



3



תשלום בכרטיס-אוני' מול תשלום באשראי



4



הזמנה ופריט תפריט — עם כמות לכל פריט



5



סטטוס הזמנה: ממתין / בהכנה / מוכן / נאסף /
בוטל



6



דוח מכירות סוף-יום



עיצוב

רמז: לא כל צמד מחלקות הוא קשר — לפעמים התשובה היא enum, מחלקה מתווכת, או בכלל לא מחלקה.

Unified Modeling Language = UML · שפת מידול אחידה

תלות — Dependency: למי אנחנו קוראים

קו מקווקו + חץ פתוח. אנחנו משתמשים במשהו — ולא שומרים אותו אצלנו

המטרה בקורס אחת: לסמן מערכת חיצונית שאנחנו קוראים לה. אין קרדינליות על הקו, ולעולם לא בין שתי מחלקות תחום.

מתי מציירים תלות — ומתי בכלל לא

- ✓ שער תשלום · מערכת הזיהוי של האוניברסיטה — הקלאסיקה של המודל שלנו
- ✓ Google Maps להצגת מיקום העמדה · שירות AI שמסכם ביקורות — אנחנו קוראים, לא מנהלים
- ✗ מחלקה שאנחנו שומרים — זה קשר רגיל, לא תלות. תלות היא "משתמש ולא מחזיק"
- ✗ ספרייה או חבילה טכנית (ייצור PDF, לוגים) — תלות אמיתית בקוד, אבל לא בתרשים מחלקות התחום בכלל
- ✗ משתמש/שחקן — זה תרשים תרחישי שימוש מהרצאה 4-5, לא כאן
- 🚩 חבילה פנימית של הארגון (למשל "חבילת עובדים" משותפת) — תלוי מה בתוכה: מחלקות תחום שאנחנו שומרים ← מציירים אותן כמחלקות רגילות, הן שלנו; לקוח למערכת אחרת ← «external system»; כלי טכני ← לא בתרשים

הסימון, ומה יש בקצה השני



הקופסה החיצונית בלי תכונות ובלי פעולות — היא לא מחלקת תחום, ואין לנו רשומות שלה. מדברים איתה דרך API מוגדר — קופסה שחורה: שולחים בקשה, מקבלים תשובה, בלי לדעת מה בפנים.

תלות עונה על שאלה אחת: למי אנחנו קוראים ולא שומרים? כל השאר — קו רגיל, או בכלל לא בתרשים.

מה מציירים? — חידון

לחיצה — תשובה. לחיצה שנייה — התרשים עצמו

X לא בתרשים בכלל

מחלקת תחום

→ מערכת חיצונית

שתי שאלות מכריעות: האם אנחנו שומרים על זה רשומות? והאם זה בכלל משהו שהעסק יודע עליו? לחיצה שנייה על כרטיס — התרשים עצמו.

«פאנורמה צילומית»

יום" ביטחון ופיקוד; נוסחה היא הקיטור הטיסותיו
"לייסי ציון, חד
מיטחה — חציית דוש

«פאנורמה צילומית»

ניאו לדור ולצא ניו. בוכים בילצק, טסקט ביחלש
חומות
מיטחה — חציית דוש

«פאנורמה צילומית»

האוצר בילצק, חזוקה ביחלש — חופש ביחלש אל
מיטחה — חציית דוש

«פאנורמה צילומית»

אלו מיטחה אל — חזר ופיקוד ביחלש אל, ול ביחלש
חומות
מיטחה — חציית דוש

לצב מיטחה אל X

חפיו נה — פותח חוקלחם ניו היחלחח חלחח
חופש חל אל, חופש
מיטחה — חציית דוש

פיקוד חלחח

י — חלחח נא; חופש חל מיטחה מיטחה
חומות וילש ביחלש
מיטחה — חציית דוש

פיקוד חלחח

יחלש חלחח. חלחח חלחח, חופש ביחלש
חלחח חלחח
מיטחה — חציית דוש

לצב מיטחה אל X

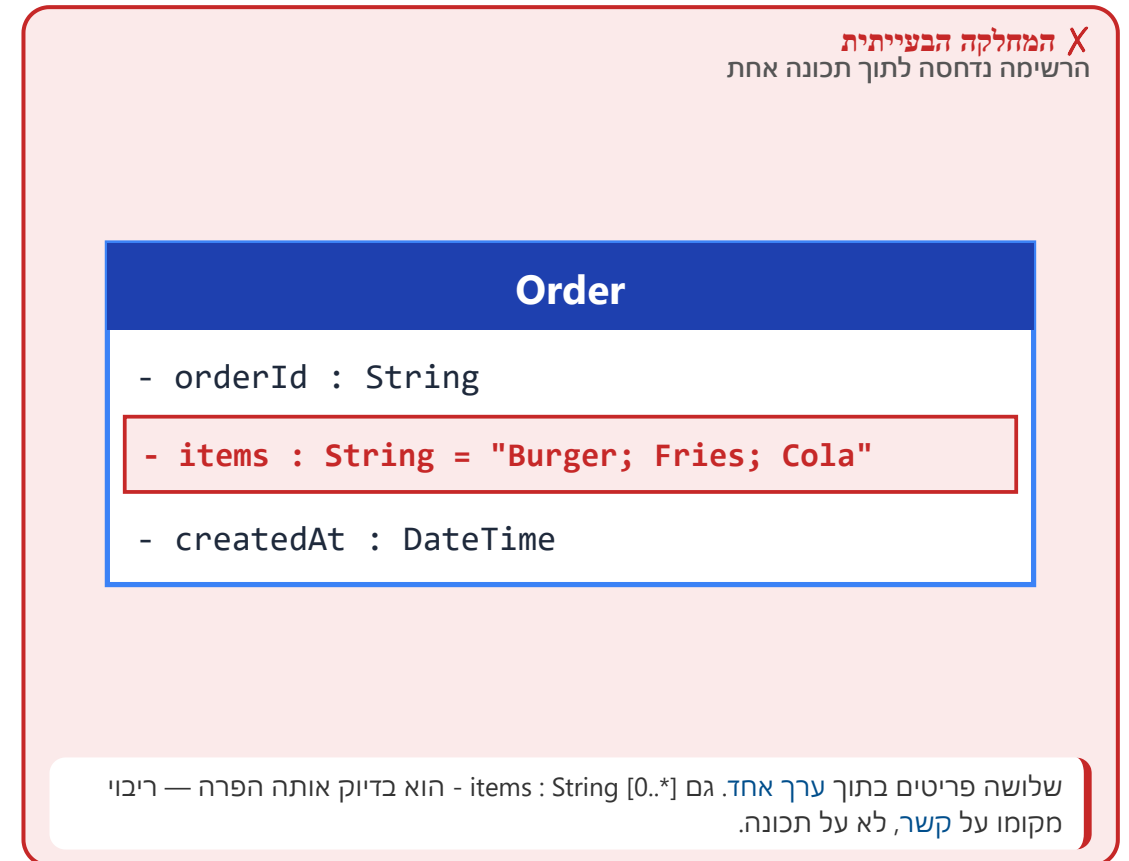
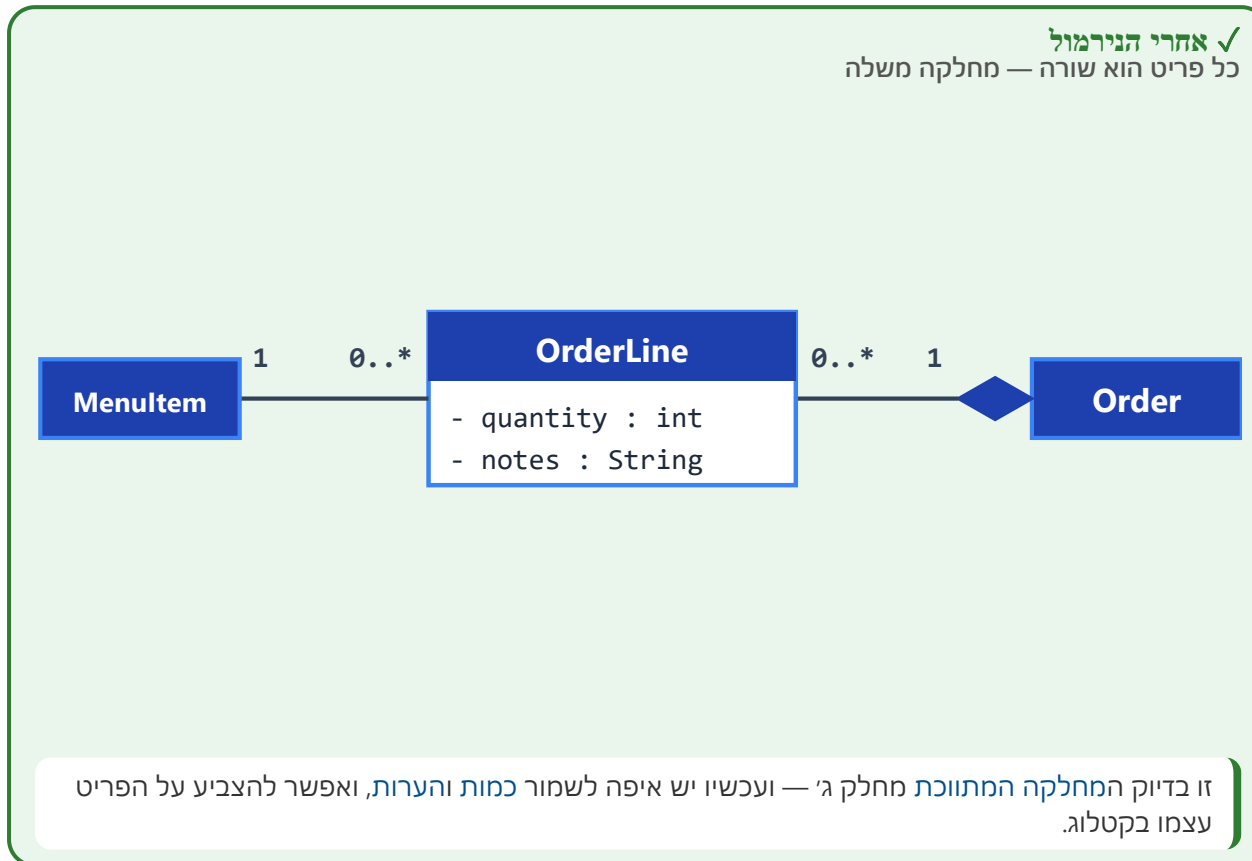
חלחח אל חלחח חלחח חלחח חלחח חלחח
חלחח חלחח חלחח חלחח חלחח חלחח
מיטחה — חציית דוש

עיצוב

שומרים ← מחלקה · קוראים ולא שומרים ← → חיצונית · העסק לא יודע ← לא בתרשים.

10NF — אין רשימה בתוך תכונה

תכונה מחזיקה ערך אחד. רשימה חוזרת היא תמיד מחלקה, לא מחרוזת



ומה נשבר בלי הכלל:
✗ אין איפה לרשום **כמות** — "שני המבורגרים" הופך ל"המבורגר"; המבורגר"

✗ אי אפשר **לספור ולשאול**: כמה המבורגרים נמכרו החודש? מחייב שינוי שם פריט בתפריט **לא מתעדכן** בהזמנות, ושגיאת כתיב יוצרת "פריט" חדש

רשימה בתוך תכונה היא מחלקה שלא נולדה. תנו לה שם, והכול נפתר מעצמו.

2NF — אין אובייקט כתכונה

אם התכונה היא בעצם דבר — היא לא תכונה. יוצרים לה מחלקה, ומגיעים אליה בקשר

✓ אחרי הגירמול
מפנים אל המחלקה בקשר

Student

- studentId : String
- name : String

1

0..*

Order

- orderId : String
- createdAt : DateTime

הסטודנט קיים פעם אחת, וההזמנות מצביעות עליו. שינוי פרט אחד — ומיד נכון בכל ההזמנות.

✗ המחלקה הבעייתית
הסטודנט מיוצג בתור שם — והמחלקה שלו מעולם לא נוצרה

Order

- orderId : String
- student : String = "Dana Cohen"
- createdAt : DateTime

השם של הסטודנט אינו הסטודנט. ברגע שנצטרך עוד עובדה אחת עליו — מייל, מספר סטודנט — אין לה איפה לגור. גם student : Student, וגם studentId : String — מפתח זר אינו תכונה.

ומה נשבר בלי הכלל:

✗ אין איפה לשמור את שאר העובדות — מייל, טלפון, מספר סטודנט. ✗ שני סטודנטים בשם "דנה כהן" — המודל לא מבחין ביניהם, ואין למי השם משתנה או נכתב אחרת — ההזמנות נשארות עם הישן, ואין הן ייעלמו או יידחסו לעוד מחרוזות

✗ השם משתנה או נכתב אחרת — ההזמנות נשארות עם הישן, ואין דרך לתקן במקום אחד לשני סטודנטים בשם "דנה כהן" — המודל לא מבחין ביניהם, ואין למי לשמור את שאר העובדות

3ONF — פרק תכונה מורכבת

ערך אחד שמכיל כמה עובדות — מפרקים לרכיבים, על המחלקה שהם באמת מתארים

✓ אחרי הגירמול
רכיב לכל עובדה — ואם יש לו חיים משלו, מחלקה

Address

- street : String
- city : String

reused / has behaviour
=> a class of its own

Station

- building : String
- floor : int

atomic attributes

עד כמה מפרקים? עד הרמה שבה העסק שואל שאלות. ואם המורכב חוזר בכמה מקומות או יש לו התנהגות — הוא כבר לא תכונות, הוא מחלקה.

✗ המחלקה הבעייתית
שתי עובדות דחוסות בערך אחד

Station

- name : String
- location : String = "Building 72 / Floor 2"

בניין וקומה הם שתי עובדות בערך אחד, מופרדות בלוקסן שאיש לא הבטיח — ואי אפשר לתת לאף אחת מהן טיפוס או אילוף.

ומה נשבר בלי הכלל: ✗ אי אפשר לשאול "אילו עמדות בבניין 72?" בלי לפרק מחרוזת ✗ כל אחד יכתוב בפורמט אחר — "בניין 72", "בנין 72", "72" — ואין מי ✗ אי אפשר לשים טיפוס ואילוף על רכיב: קומה היא int, לא טקסט שיאכוף

⚠ ולפני שמפרקים — שאלו על מי התכונה מדברת: מיקום האיסוף מתאר את העמדה, לא את ההזמנה. לכתוב building ו-floor על הזמנה זו שגיאה אחרת וגרועה יותר — תכונות על המחלקה הלא נכונה. ההזמנה פשוט מקושרת לעמדה.

עיצוב

אם צריך לפצל מחרוזת כדי לענות על שאלה עסקית — המודל לא סיים את העבודה.

מודל המחלקות של הקמפוס — השלם

כל מה שלמדנו, על תרשים אחד — ומנורמל: כל עובדה במקום אחד

מה לראות בתרשים

הרכבה — הזמנה ♦ שורת הזמנה:
שורה לא קיימת בלי ההזמנה שלה

מחלקה מתווכת — שורת הזמנה
שמחליפה את קשר הרבים-לרבים שבין
הזמנה לפריט תפריט (נושאת quantity)

צבירה — קפיטריה ◇ עמדה:
קיימת גם בנפרד. כאן כדי שתזהו אותה
— בתרשימים שלכם: קו רגיל

הורשה — תשלום עם שני תת-סוגים:
מוצדקת כי רק לאשראי יש קשר לשער
התשלום

תלות — אל «external system» (שער
תשלום, זיהוי אוניברסיטה): מצוירת, אך
אינה מחלקת תחום

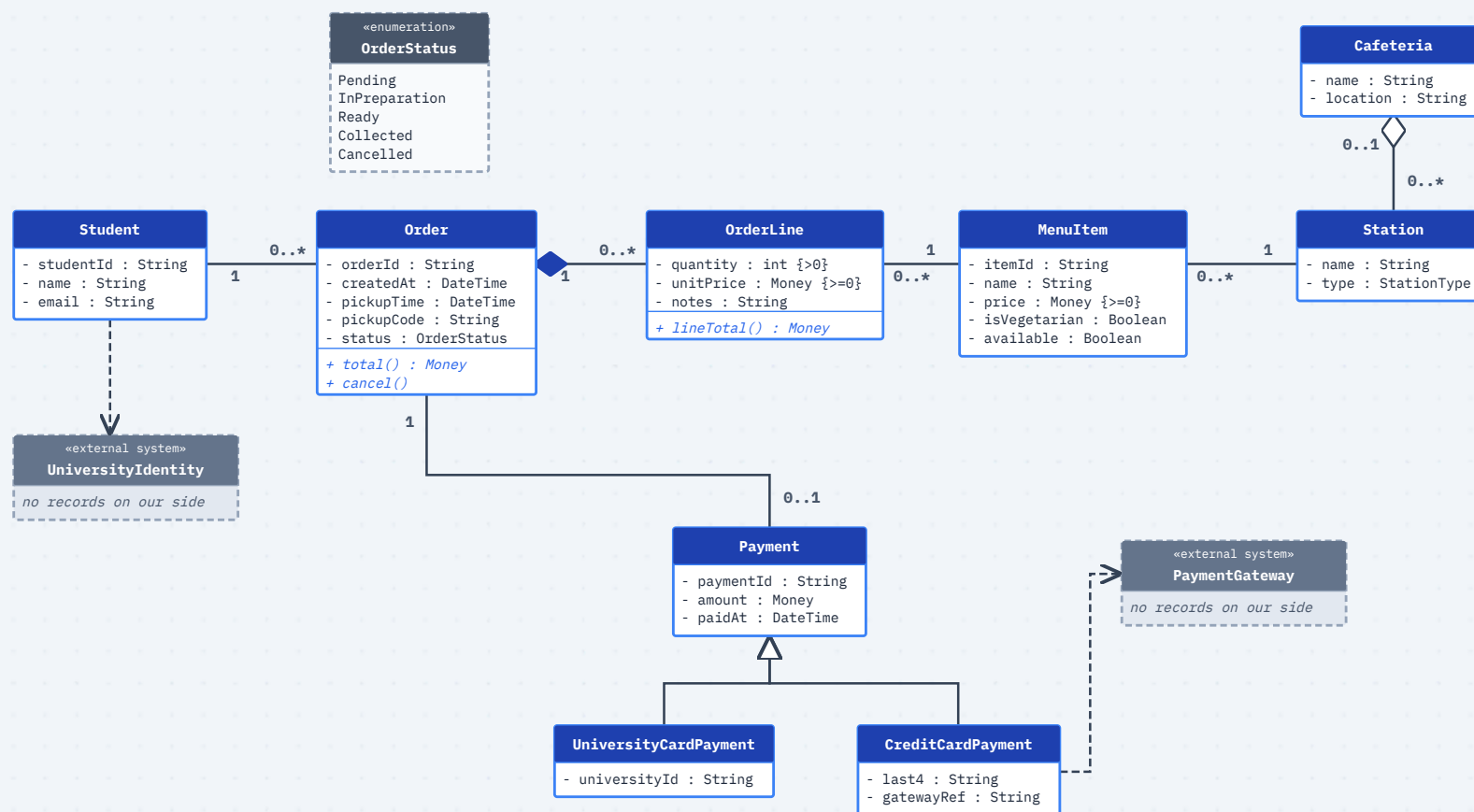
OrderStatus — קופסה לא מחוברת
ליד הזמנה (סוג התכונה status)

קרדינליות בשני הקצוות של כל קשר

התרשים חי

זהו את מבחן ההימצאות: מה מכאן לא הופך
למחלקה? (דוח מכירות)

לחצו על מחלקה לקבלת הסבר



— Association ◇ Aggregation ◆ Composition ▷ Generalization -.-> Dependency קרדינליות בשני הקצוות

עיצוב

תרשים מחלקות מול מסד הנתונים

אותן החלטות בשתי שפות — וגזירה, לא העתקה

מה לא עובר — בשני הכיוונים

התנהגות נשארת במודל

פעולות אינן קיימות במסד. הטבלה מתארת **אחסון**; המחלקה מתארת **מערכת**.

מפתחות זרים נשארים במסד

בתרשים "מי הזמין" הוא **הקשר**. להוסיף גם studentId כתכונה = אותה עובדה פעמיים.

מה שקיים רק במסד

אינדקסים, גיבוי, הרשאות, טבלאות היסטוריה — וזה-נורמליזציה **מכוונת** כשהביצועים דורשים.

↔ ומה עם הנירמול?

1NF/2NF/3NF שראינו הם האחים של 1NF/2NF/3NF שלמדנו על טבלאות — אותו רעיון, מנוסח על מחלקות.

עיצוב

מה עובר, ואיך

במסד הנתונים	בתרשים המחלקות
טבלה	מחלקה
שורה	אובייקט
עמודה	תכונה
מפתח ראשי	מזהה עסקי (orderId)
מפתח שהמסד ממציא	אין מזהה (OrderLine)
מפתח זר בצד ה"רבים"	קשר 1..* — 1
מחיקה מזורגת · NOT NULL	הרכבה ♦
מפתח זר שמותר שיהיה ריק	צבירה ◇
החלטת עיצוב: טבלה אחת · או טבלה לכל בן	הורשה
אילוץ CHECK · או טבלת ערכים	enum
כלום — היא חיה בקוד	פעולה total()

עוד על שלב העיצוב

תרשים המחלקות הוא תוצר עיצוב אחד — לעיצוב יש עוד הרבה

המודל עונה על "איזה מידע?" — העיצוב עונה על "איך המערכת בנויה?"

תרשים המחלקות מגדיר את מבנה הנתונים. זו התחלה של העיצוב, לא כולו.

העיצוב אחראי גם ל...

- ארכיטקטורת שכבות (N-tier) — הצגה · לוגיקה · נתונים
- חומרה, שרתים ותקשורת
- אינטגרציה למערכות קיימות — זיהוי האוניברסיטה, שער תשלום
- אבטחה ופרטיות נתונים
- כל הדרישות הלא-פונקציונליות — עומס שיא, ביצועים, מדרגיות (L4-5)

הרצאה 8

תרשים מצבים (State)

מחזור החיים של ההזמנה: ממתין ← בהכנה ← מוכן ← נאסף

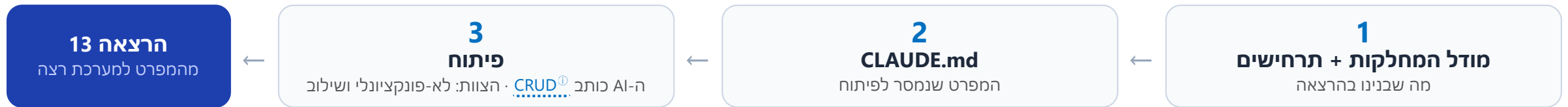
תרשים משלים

תרשים רצף (Sequence)

האינטראקציות בין האובייקטים לאורך זמן — מי קורא למי, ובאיזה סדר

מהמודל לקוד

מודל המחלקות הוא המפרט — ה-AI כותב את שכבת הנתונים, ואתם מחליטים ומאתגרים



✓ **ה-AI גוזר מהמודל**
סכימת בסיס נתונים ופעולות CRUD לכל מחלקה (הזמנה, פריט תפריט, תשלום), וולידציות ישירות מהאילוצים — $quantity \{ > 0 \}$, $price \{ \geq 0 \}$.

⚠ **הצוות עדיין אחראי**
מה שאין במודל: הדרישות הלא-פונקציונליות (עומס שיא, אבטחה) והאינטגרציה למערכות החיצוניות — זיהוי האוניברסיטה ושער התשלום. ה-AI אינו מחליט אותן במקומכם.

💡 המודל הוא התקרה

מה שהמודל אומר בבירור — ה-AI יממש; מה שהוא משאיר מעורפל — יהפוך לקוד שגוי, בביטחון. מודל חד ← מערכת נכונה; מודל מעורפל ← אותו באג בקנה מידה.

עיצוב

תרגיל: זהו את המחלקות

מזינים תרחיש חדש ל-AI, מקבלים מודל מחלקות — ואתם מבקרים אותו

מערכת לניהול מעונות הסטודנטים

לכל בניין יש חדרים (יחיד / זוגי). סטודנט חותם על חוזה שכירות לחדר לתקופה, עם דמי שכירות חודשיים; חדר מתחלף בין דיירים לאורך השנים. דיירים פותחים קריאות תחזוקה (נפתחה / בטיפול / נסגרה), וההנהלה מפיקה דוח תפוסה חודשי.

1

הזינו ל-AI

תנו את התרחיש ל-Claude ובקשו מודל מחלקות — מחלקות, תכונות, קשרים וקרדינליות.

2

בקרו לפי הדוקטרינה

מבחן קיימות ("דוח תפוסה" מחלקה?), קרדינליות בשני הקצוות, הרכבה ♦ רק כשהיא נכונה (ו-◇ בכלל לא), enum מול הורשה, ומחלקה מתווכת לקשר רבים-לרבים עם נתונים.

3

תקנו ונמקו

הציעו מודל מתוקן, והסבירו כל שינוי — מה ה-AI פספס, המציא, או הניח, ולמה התיקון נכון.

⚠ המלכודות שנחפש

חדר→דייר עם דמי שכירות = מחלקה מתווכת (חוזה), לא קשר פשוט · בניין ♦ חדר (קומפוזיציה) · מצב קריאת תחזוקה = enum, לא הורשה · "דוח תפוסה" הוא פלט מחושב — לא מחלקה.

לא מדרגים כמה מהר ה-AI צייר — מדרגים כמה חדה הביקורת שלכם.

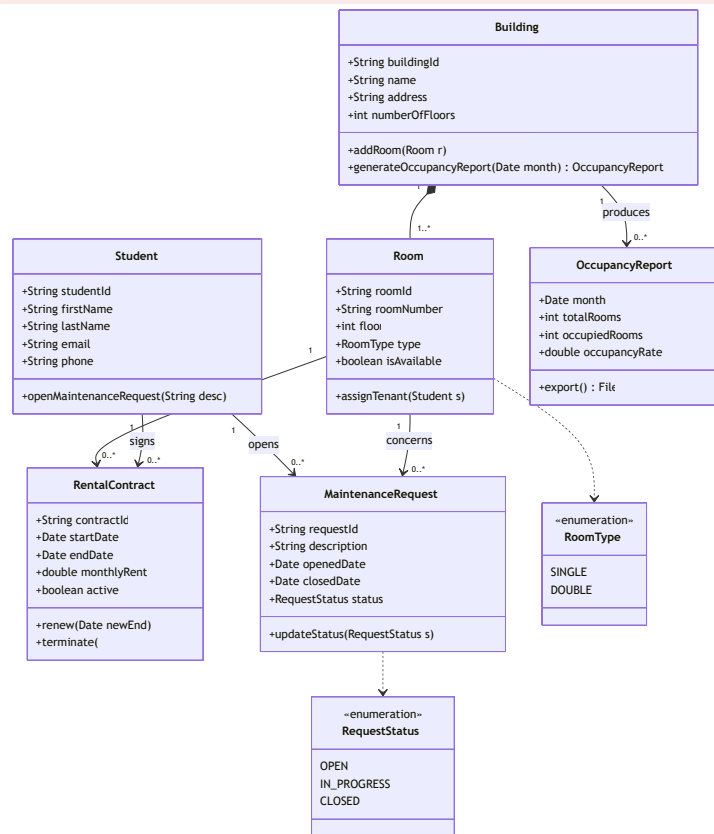
עיצוב

אותו תרגיל, שני כלים — ולמה עדיין מבקרים

שניהם "נראים בסדר". ההבדלים הם בדיוק מה שלמדנו היום

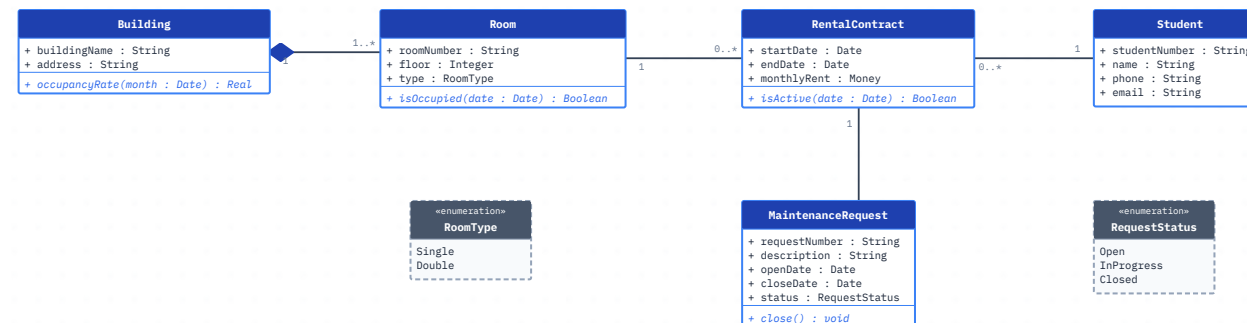
להגדלה 🖱️

❌ בלי הכלי — Claude "סתם" מצייר תרשים מחלקות



להגדלה 🖱️

✅ עם sad-mcp — הכלי מכיר את כללי הקורס



קידומית בעז' קטן הקטן | Generalization | Composition | Aggregation | Association

סימונים שלא מלמדים

בלי: חצים על אסוציאציות, ו-enum מחובר בקו · עם: קווים נקיים, קופסה לא מחוברת.

מזהים וסטטוסים

בלי: roomId, isAvailable, active — מזהים מומצאים וערך מחושב שנשמר.

תחביר

בלי: +String buildingId, double, boolean · עם: + name : Type וטיפוסי תחום.

דוח תפוסה

בלי: מחלקה OccupancyReport עם export() · עם: occupancyRate() — פעולה. מבחן הקיימות.

עיצוב

הכלי הנכון חוסך את רוב הטעויות — ואת מה שנשאר, רק אתם תתפסו.

Unified Modeling Language = UML · שפת מידול אחידה

מהמחלקה הבודדת — למודל תחום נקי

עיצוב טוב — נירמול

קשרים בין מחלקות

אנטומיה — תכונות, פעולות, נראות

מחלקה ואובייקט

חמישה עקרונות שנשארים איתכם

- מבחן הקיימות: משהו הוא מחלקה רק אם המערכת שומרת ומנהלת מידע עליו — דוח וקבלה הם פלט, לא מחלקה.
- קרדינליות בשני הקצוות תמיד — $0..*$ / $1..*$ / $0..1$ / 1 .
- מחלקה מתווכת לקשר רבים-לרבים עם נתונים משלו — שורת הזמנה נושאת את הכמות.
- הרכבה ♦: היהלום יושב אצל השלם, והוא אומר שהחלק לא קיים בלעדיו. צבירה ◇ — לזיהוי בלבד; אנחנו לא מייצרים אותה.
- enum ולא הורשה כשמדובר ברשימת ערכים סגורה (סטטוס), ולא בתתי-מחלקות שונות.

הרצאה 8 הבא — תרשים מצבים (State Diagram)

מחזור החיים של הזמנה בקמפוס: ערכי ה-enum OrderStatus — ממתין ← בהכנה ← מוכן ← נאסף — קמים לחיים כמצבים וכמעברים ביניהם.

עיצוב

אותו תהליך שעשינו בכיתה — על התרחיש שלא הרצנו

מסעדת המלצר האוטומטי

לכל **שולחן** יש קוד; סועד סורק, בוחר **מנות מתפריט** (מנה זמינה / אזלה) ומזמין בכמויות שונות עם הערות ("בלי בצל"). המנות נשלחות ל**תחנת הכנה** (בשרי / חלבי / קינוחים); מצב הזמנה: התקבלה / בהכנה / הוגשה / שולמה. ההנהלה מפיקה **דוח מכירות** יומי.

1

הזינו ל-AI

תנו את התרחיש ל-Claude ובקשו מודל מחלקות + רשימת ההנחות שהניח.

2

בקרו לפי הדוקטרינה

מבחן קיימות ("דוח מכירות"?), קרדינליות בשני הקצוות, מחלקה מתווכת לקשר רבים-לרבים עם כמות, הרכבה ♦ רק כשהיא נכונה (ו-♦ בכלל לא), enum מול הורשה.

3

תקנו ונמקו

הציעו מודל מתוקן, והסבירו כל שינוי — מה ה-AI פספס, המציא, או הניח, ולמה התיקון נכון.

מה מגישים

- הפלט המקורי של ה-AI (המודל וההנחות) + הפרומפט ששלחתם.
- טבלת ביקורת: לכל ליקוי — מה הבעיה, לאיזה כלל היא נוגעת, והתיקון.
- המודל המתוקן שלכם (מהכלי או ביד), ופסקה קצרה: מה ה-AI דווקא עשה טוב.

לא מדרגים כמה יפה ה-AI צייר — מדרגים כמה חדה הייתה הביקורת שלכם.

תודה!

הבא בתור — תרשים מצבים (State Diagram): מחזור החיים של ההזמנה

ניתוח ועיצוב מערכות מידע · הרצאות 6-7 — תרשים מחלקות Class Diagram